EuroPython
Society

TENANTS OF
PLANET SCALE SYSTEMS
WITH PYTHON

Abhimanyu Singh Shekhawat

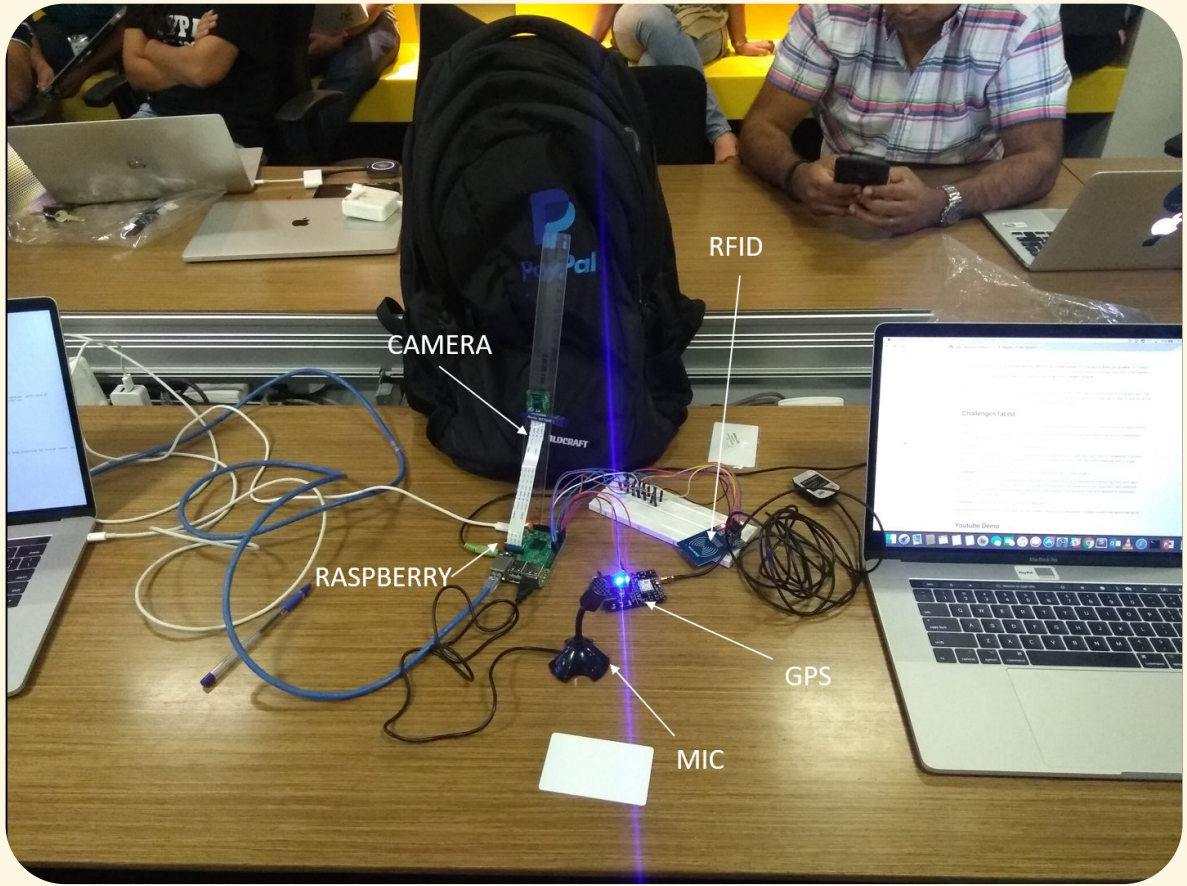
Hi! I am Abhimanyu

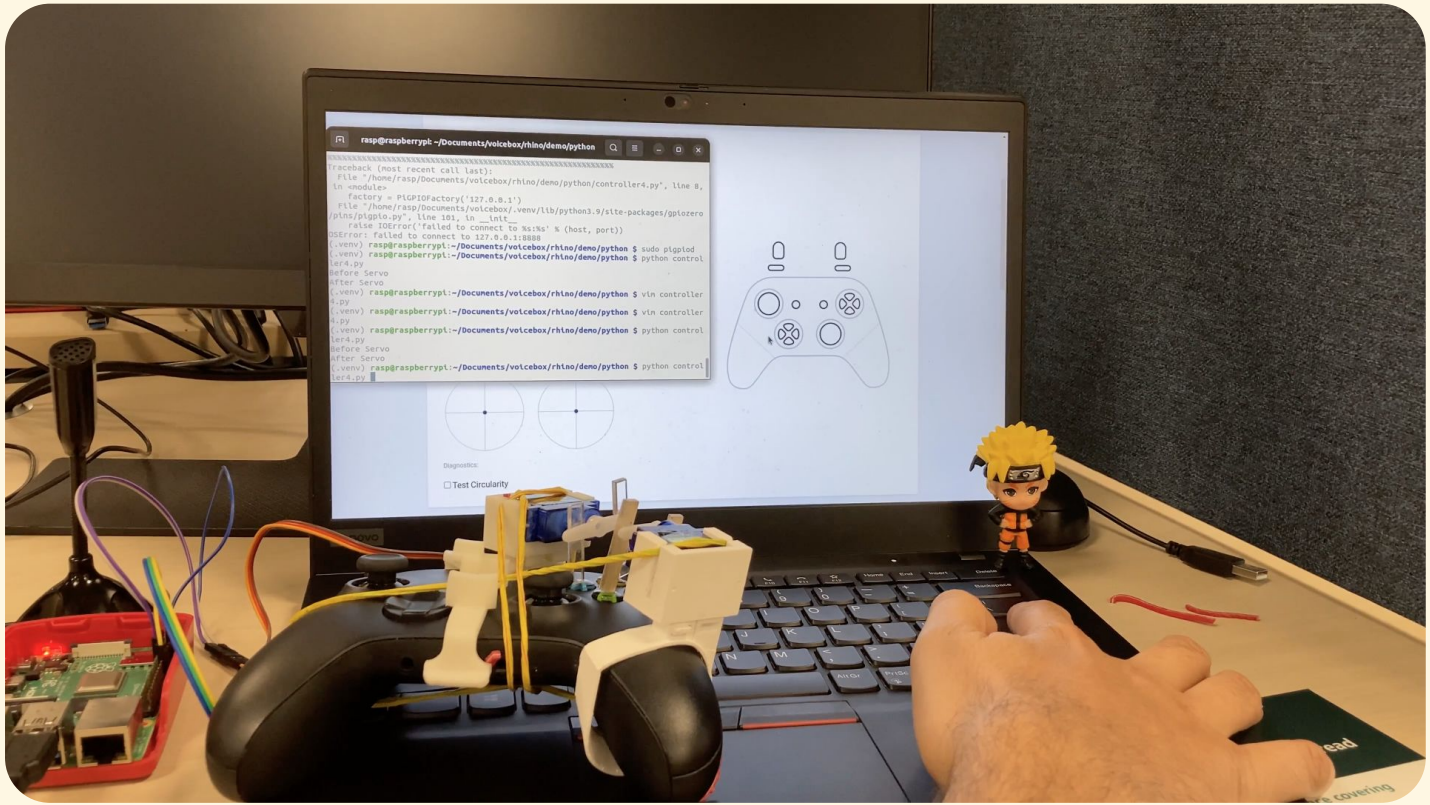
Building Distributed Databases for 8 Years

@abhimanyubitsgoa
@abshekha

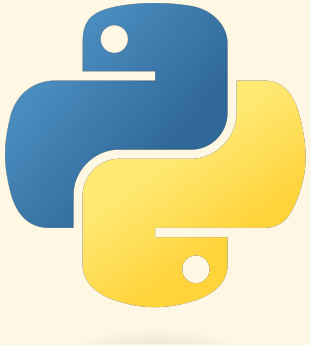
Hi! I am Abhimanyu

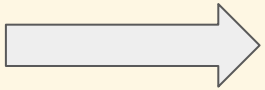
PLAYING WITH PYTHON SINCE FOREVERRR!


RFID
CAMERA
RASPBERRY
GPS
MIC


rasp@raspberrypi: ~/Documents/voicebox/rhino/demo/python
Traceback (most recent call last):
File "/home/rasp/Documents/voicebox/rhino/demo/python/controller4.py", line 8, in <module>
factory = PiGPIOFactory('127.0.0.1')
File "/home/rasp/Documents/voicebox/.venv/lib/python3.9/site-packages/gpiozero/pins/pigpio.py", line 101, in __init__
raise IOError('failed to connect to %s:%s' % (host, port))
OSError: failed to connect to 127.0.0.1:8888
(.venv) rasp@raspberrypi:~/Documents/voicebox/rhino/demo/python $ sudo pigpiod
(.venv) rasp@raspberrypi:~/Documents/voicebox/rhino/demo/python $ python controller4.py
Before Servo
After Servo
(.venv) rasp@raspberrypi:~/Documents/voicebox/rhino/demo/python $ vim controller4.py
(.venv) rasp@raspberrypi:~/Documents/voicebox/rhino/demo/python $ vim controller4.py
(.venv) rasp@raspberrypi:~/Documents/voicebox/rhino/demo/python $ python controller4.py
Before Servo
After Servo
(.venv) rasp@raspberrypi:~/Documents/voicebox/rhino/demo/python $ python controller4.py
Diagnostics
Test Circularity


3:30 PM
Abhimanyu Shekhawat (Presenting, annotating)
Anatomy of Agent Harness

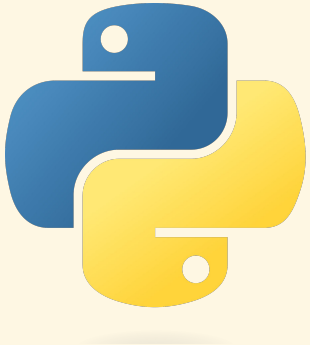


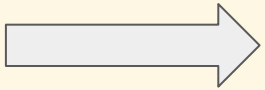


T

D

D





TRANSFORMATION DRIVEN DEVELOPMENT


db = {}

MILLIONS

MILLIONS

TOGETHER

MILLIONS

TOGETHER

IDENTICAL

MILLIONS

TOGETHER

IDENTICAL

99.999%

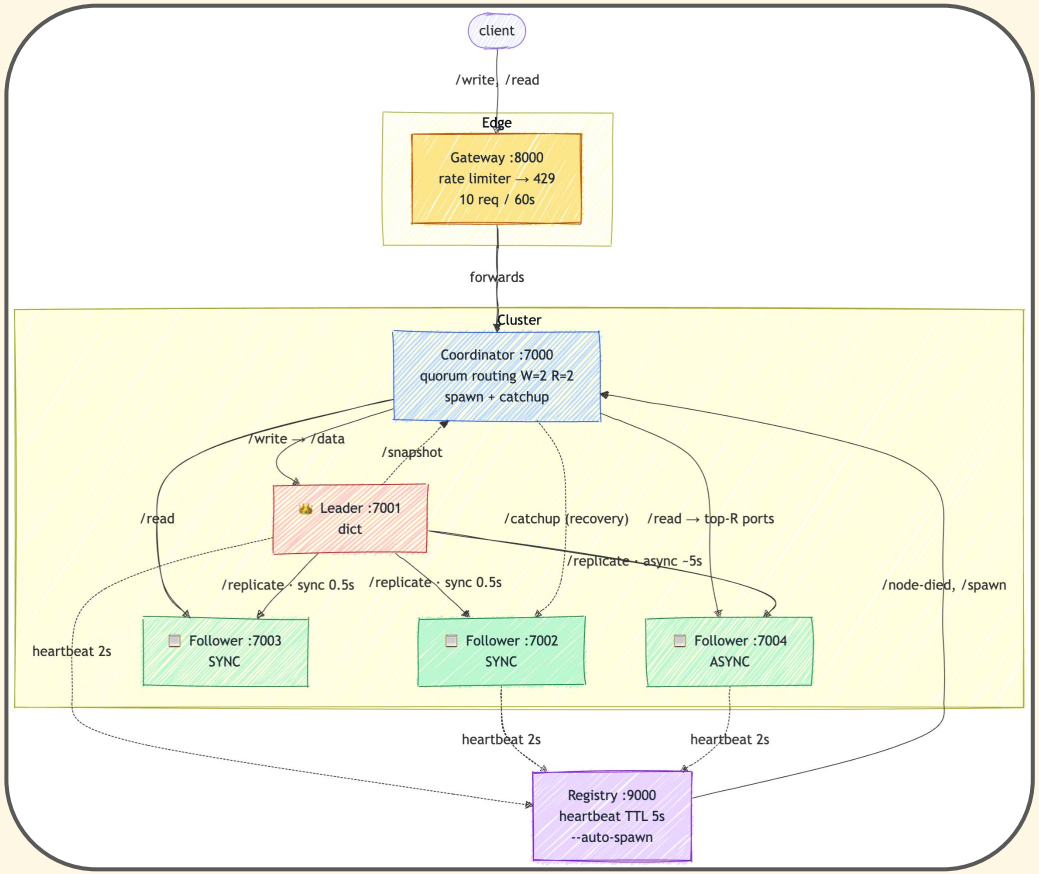
MILLIONS

TOGETHER

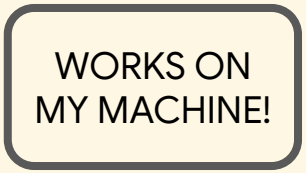
IDENTICAL

99.999% [5.256 MIN/YEAR]

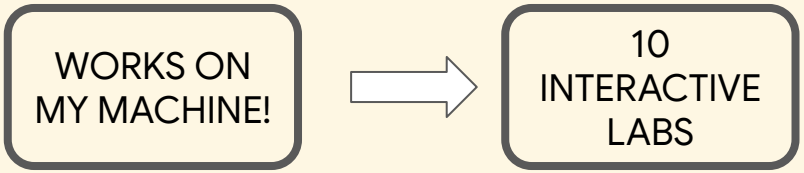

db = {}


client
/write, /read
Edge
Gateway :8000
rate limiter → 429
10 req / 60s
forwards
Cluster
Coordinator :7000
quorum routing W=2 R=2
spawn + catchup
/write → /data
/snapshot
/read
Leader :7001
dict
/catchup (recovery)
/read → top-R ports
/replicate - async ~5s
/replicate - sync 0.5s
/replicate - sync 0.5s
/node-died, /spawn
heartbeat 2s
Follower :7003
SYNC
Follower :7002
SYNC
Follower :7004
ASYNC
heartbeat 2s
heartbeat 2s
Registry :9000
heartbeat TTL 5s
--auto-spawn

WORKSHOP PLAN

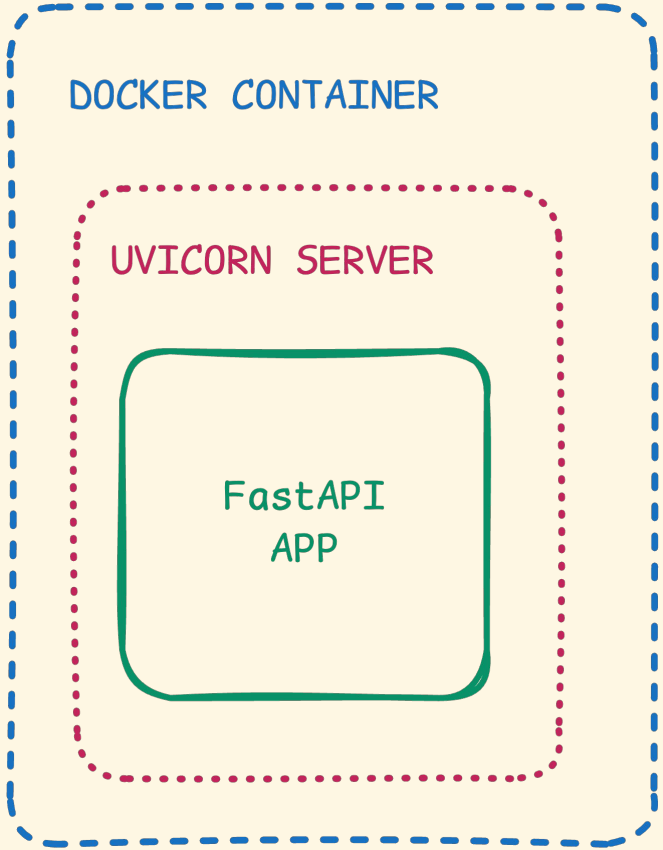


WORKSHOP PLAN



WORKSHOP PLAN

LAB SETUP



SETUP CHECK

```
root@56710772e3f0:/workspace/build-kvstore# make verify

Build-a-KVStore setup check
---------------------------
  [OK]   running inside the workshop container
  [OK]   python found (Python 3.12.3)
  [OK]   workshop libraries import (fastapi, uvicorn, requests, httpx)
  [OK]   curl found
  [OK]   make found
  [OK]   tmux found
  [OK]   tmux can create a session (the 'make lab' dashboard will work)
  [OK]   stage-01 node booted and answers /health on :5001
  [OK]   HTTP write + read round-trip works (the store stores)

  ┌──────────────────────────────────────────────┐
  │   SETUP VERIFIED — you are ready to build    │
  │   a distributed KV store. See you there!     │
  └──────────────────────────────────────────────┘

  (If the box above looks like underscores, use Windows Terminal / iTerm.)
```

LAB CHEAT SHEET

```
make todo STAGE=NN      # make todo STAGE=03
```

LAB CHEAT SHEET

```
make todo STAGE=NN      # make todo STAGE=03
```

```
make checkpoint STAGE=NN      # make checkpoint STAGE=03
```

LAB CHEAT SHEET

```
make todo STAGE=NN      # make todo STAGE=03
```

```
make checkpoint STAGE=NN      # make checkpoint STAGE=03
```

```
make lab STAGE=NN      # make lab STAGE=03
```

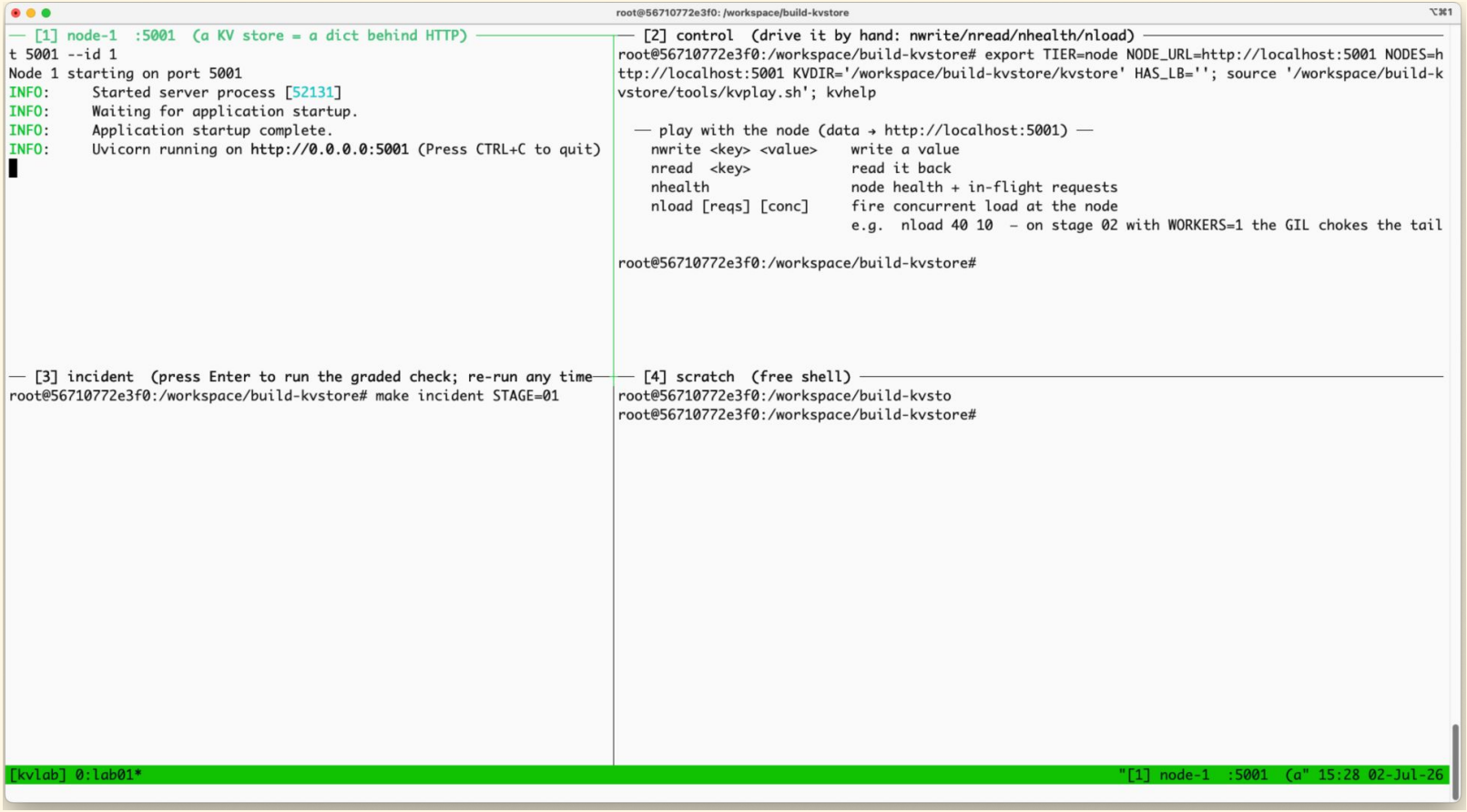
LAB CHEAT SHEET

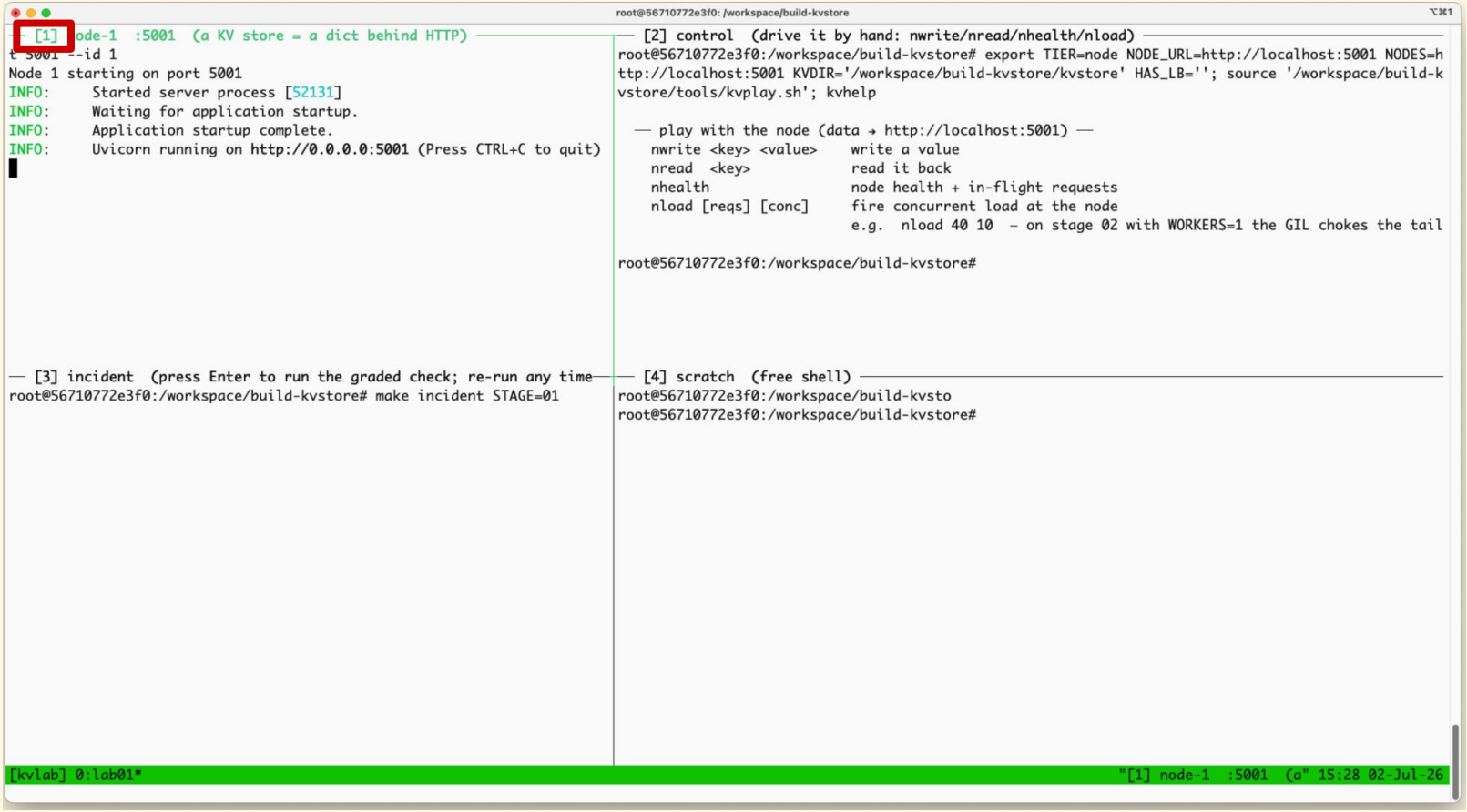
```
make todo STAGE=NN      # make todo STAGE=03
```

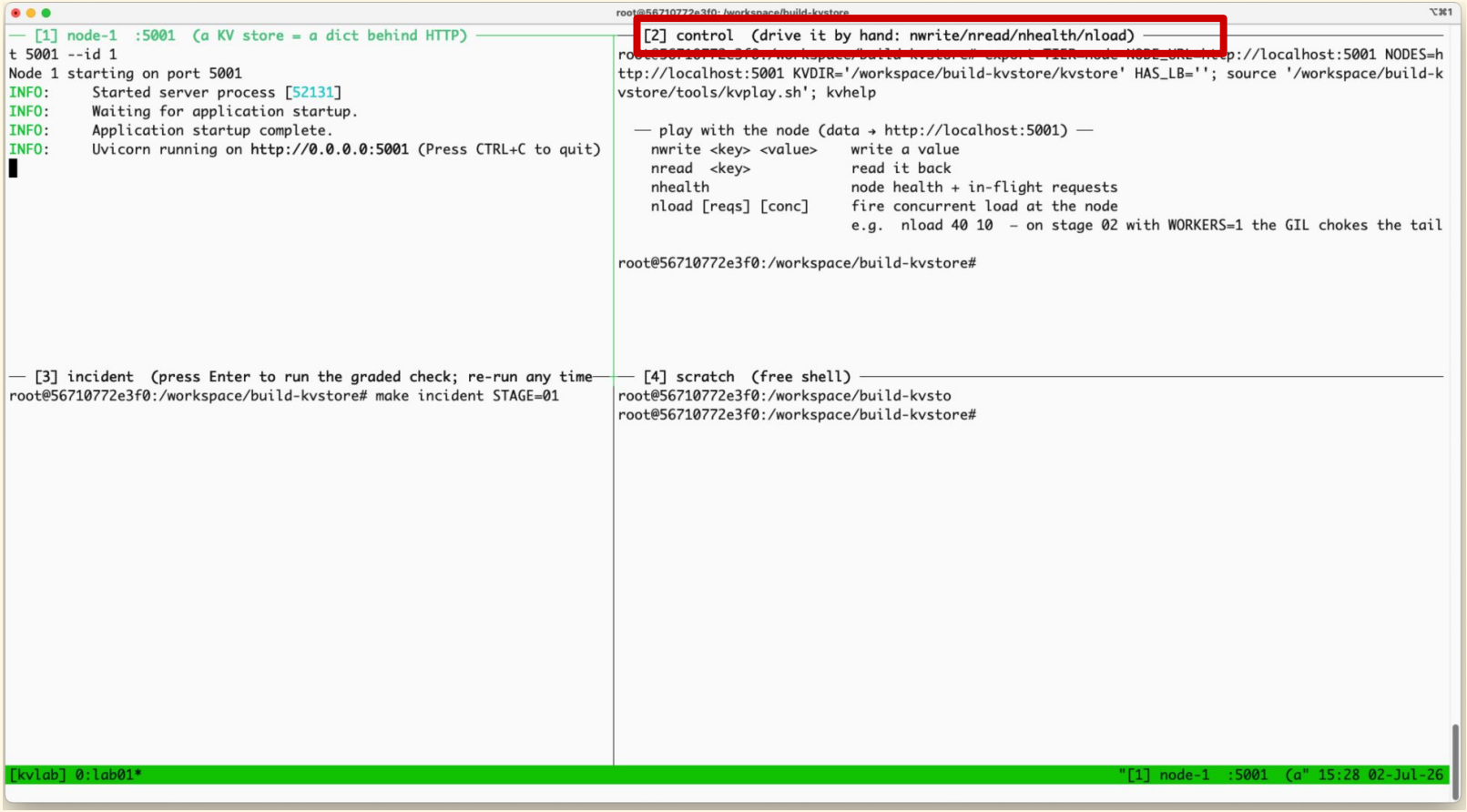
```
make checkpoint STAGE=NN      # make checkpoint STAGE=03
```

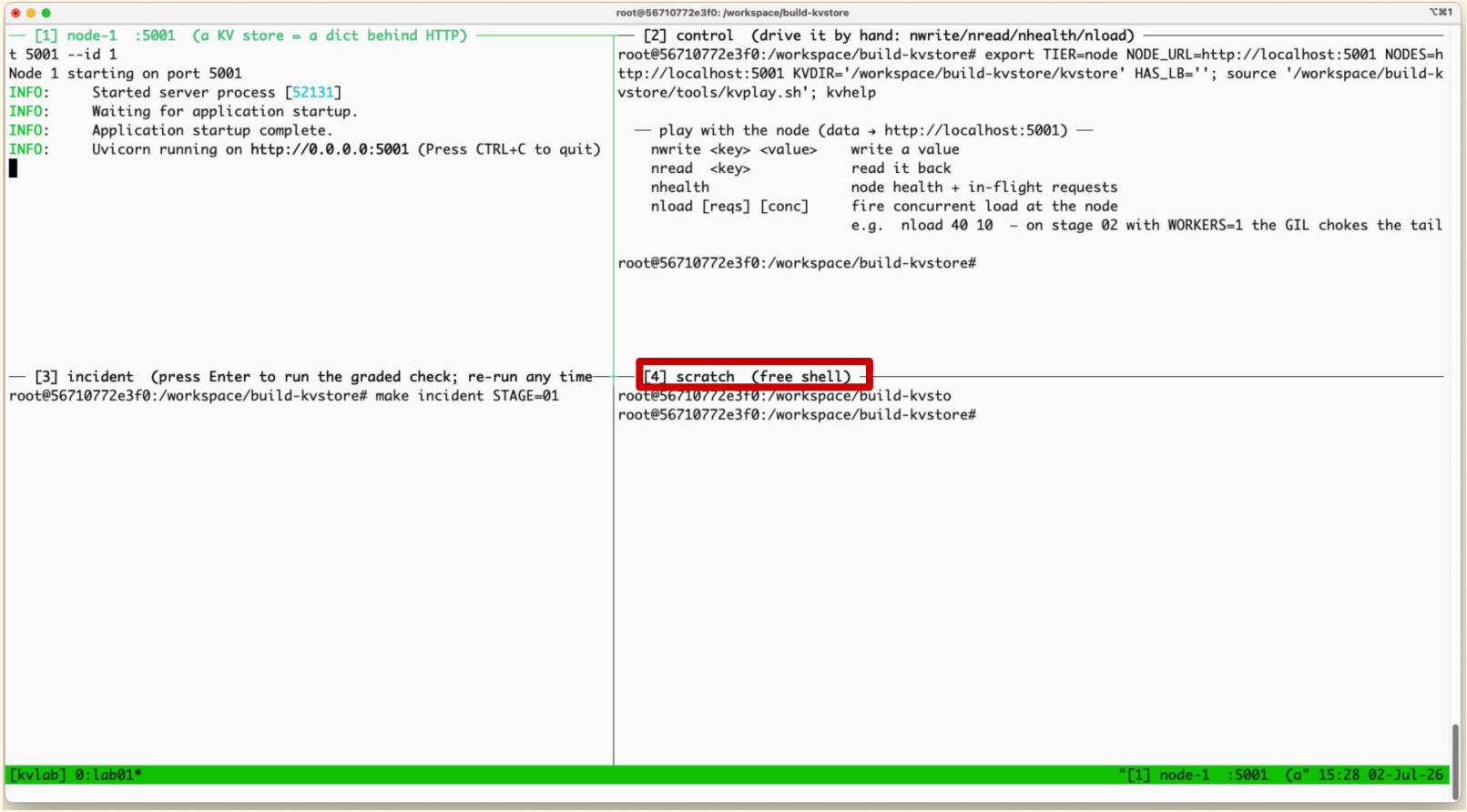
```
make lab STAGE=NN      # make lab STAGE=03
```

```
make incident STAGE=NN      # make incident STAGE=03
```


root@56710772e3f0: /workspace/build-kvstore
[1] node-1 :5001 (a KV store = a dict behind HTTP)
t 5001 --id 1
Node 1 starting on port 5001
INFO: Started server process [52131]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5001 (Press CTRL+C to quit)
[2] control (drive it by hand: nwrite/nread/nhealth/nload)
root@56710772e3f0:/workspace/build-kvstore# export TIER=node NODE_URL=http://localhost:5001 NODES=http://localhost:5001 KVDIR='/workspace/build-kvstore/kvstore' HAS_LB=''; source '/workspace/build-kvstore/tools/kvplay.sh'; kvhelp
play with the node (data → http://localhost:5001)
nwrite <key> <value> write a value
nread <key> read it back
nhealth node health + in-flight requests
nload [reqs] [conc] fire concurrent load at the node
e.g. nload 40 10 - on stage 02 with WORKERS=1 the GIL chokes the tail
root@56710772e3f0:/workspace/build-kvstore#
[3] incident (press Enter to run the graded check; re-run any time
root@56710772e3f0:/workspace/build-kvstore# make incident STAGE=01
[4] scratch (free shell)
root@56710772e3f0:/workspace/build-kvsto
root@56710772e3f0:/workspace/build-kvstore#
[kvlab] 0:lab01*
"[1] node-1 :5001 (a" 15:28 02-Jul-26


root@56710772e3f0: /workspace/build-kvstore
[1] node-1 :5001 (a KV store = a dict behind HTTP)
t 5001 --id 1
Node 1 starting on port 5001
INFO: Started server process [52131]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5001 (Press CTRL+C to quit)
[2] control (drive it by hand: nwrite/nread/nhealth/nload)
root@56710772e3f0:/workspace/build-kvstore# export TIER=node NODE_URL=http://localhost:5001 NODES=http://localhost:5001 KVDIR='/workspace/build-kvstore/kvstore' HAS_LB=''; source '/workspace/build-kvstore/tools/kvplay.sh'; kvhelp
play with the node (data → http://localhost:5001)
nwrite <key> <value> write a value
nread <key> read it back
nhealth node health + in-flight requests
nload [reqs] [conc] fire concurrent load at the node
e.g. nload 40 10 - on stage 02 with WORKERS=1 the GIL chokes the tail
root@56710772e3f0:/workspace/build-kvstore#
[3] incident (press Enter to run the graded check; re-run any time
root@56710772e3f0:/workspace/build-kvstore# make incident STAGE=01
[4] scratch (free shell)
root@56710772e3f0:/workspace/build-kvsto
root@56710772e3f0:/workspace/build-kvstore#
[kvlab] 0:lab01*
"[1] node-1 :5001 (a" 15:28 02-Jul-26


[1] node-1 :5001 (a KV store = a dict behind HTTP)
t 5001 --id 1
Node 1 starting on port 5001
INFO: Started server process [52131]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5001 (Press CTRL+C to quit)
[2] control (drive it by hand: nwrite/nread/nhealth/nload)
ttp://localhost:5001 KVDIR='/workspace/build-kvstore/kvstore' HAS_LB=''; source '/workspace/build-k
vstore/tools/kvplay.sh'; kvhelp
play with the node (data → http://localhost:5001)
nwrite <key> <value> write a value
nread <key> read it back
nhealth node health + in-flight requests
nload [reqs] [conc] fire concurrent load at the node
e.g. nload 40 10 - on stage 02 with WORKERS=1 the GIL chokes the tail
root@56710772e3f0:/workspace/build-kvstore#
[3] incident (press Enter to run the graded check; re-run any time
root@56710772e3f0:/workspace/build-kvstore# make incident STAGE=01
[4] scratch (free shell)
root@56710772e3f0:/workspace/build-kvsto
root@56710772e3f0:/workspace/build-kvstore#
[kvlab] 0:lab01*
"[1] node-1 :5001 (a" 15:28 02-Jul-26


root@56710772e3f0: /workspace/build-kvstore
[1] node-1 :5001 (a KV store = a dict behind HTTP)
t 5001 --id 1
Node 1 starting on port 5001
INFO: Started server process [52131]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5001 (Press CTRL+C to quit)
[2] control (drive it by hand: nwrite/nread/nhealth/nload)
root@56710772e3f0:/workspace/build-kvstore# export TIER=node NODE_URL=http://localhost:5001 NODES=http://localhost:5001 KVDIR='/workspace/build-kvstore/kvstore' HAS_LB=''; source '/workspace/build-kvstore/tools/kvplay.sh'; kvhelp
play with the node (data → http://localhost:5001)
nwrite <key> <value> write a value
nread <key> read it back
nhealth node health + in-flight requests
nload [reqs] [conc] fire concurrent load at the node
e.g. nload 40 10 - on stage 02 with WORKERS=1 the GIL chokes the tail
root@56710772e3f0:/workspace/build-kvstore#
[3] incident (press Enter to run the graded check; re-run any time
root@56710772e3f0:/workspace/build-kvstore# make incident STAGE=01
[4] scratch (free shell)
root@56710772e3f0:/workspace/build-kvsto
root@56710772e3f0:/workspace/build-kvstore#
[kvlab] 0:lab01*
"[1] node-1 :5001 (a" 15:28 02-Jul-26

LAB CHEAT SHEET

```
make todo STAGE=NN      # make todo STAGE=03
```

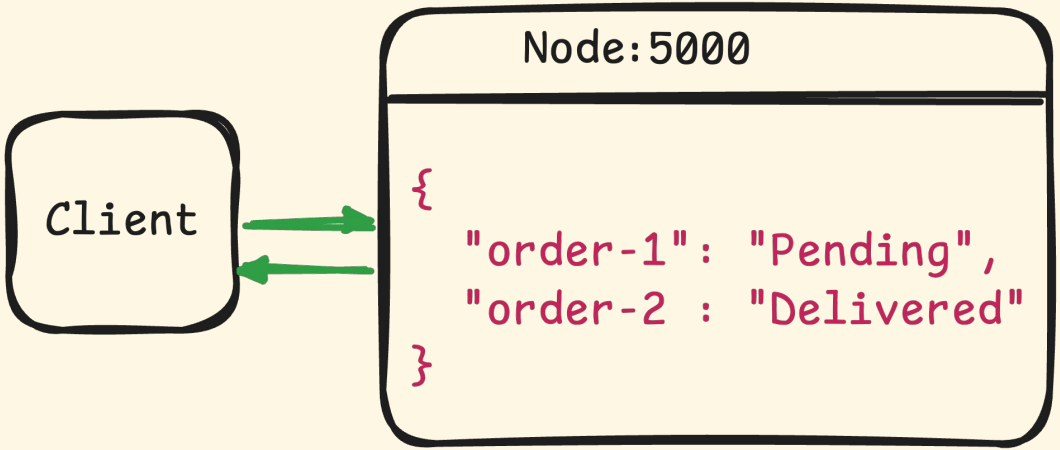
```
make checkpoint STAGE=NN      # make checkpoint STAGE=03
```

```
make lab STAGE=NN      # make lab STAGE=03
```

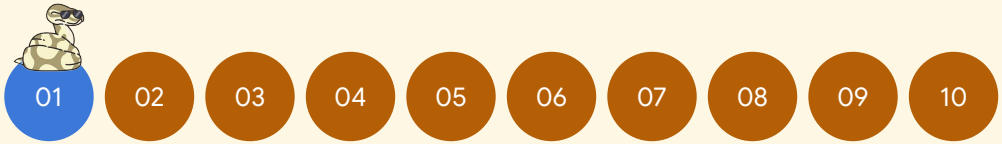
SINGLE NODE

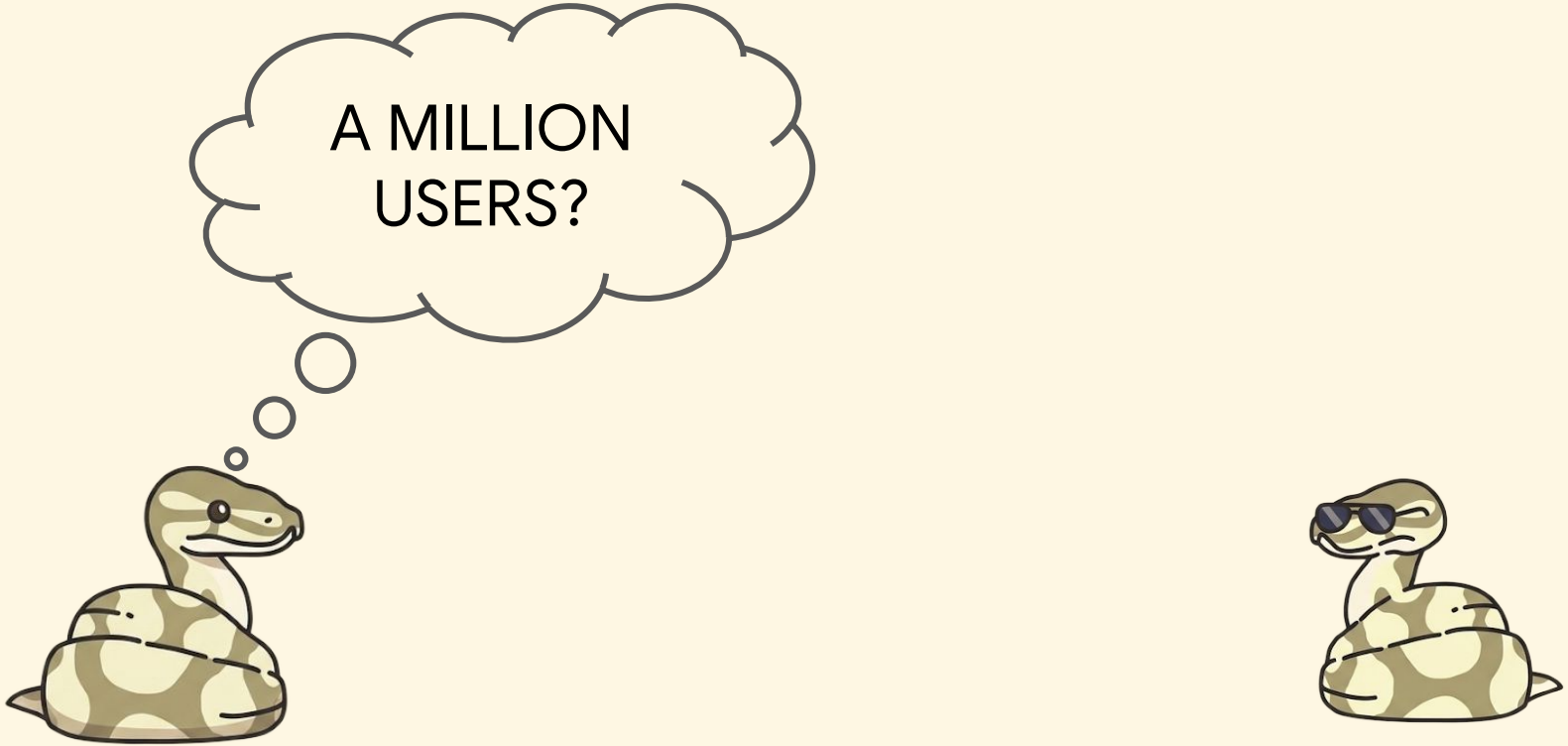
01 02 03 04 05 06 07 08 09 10

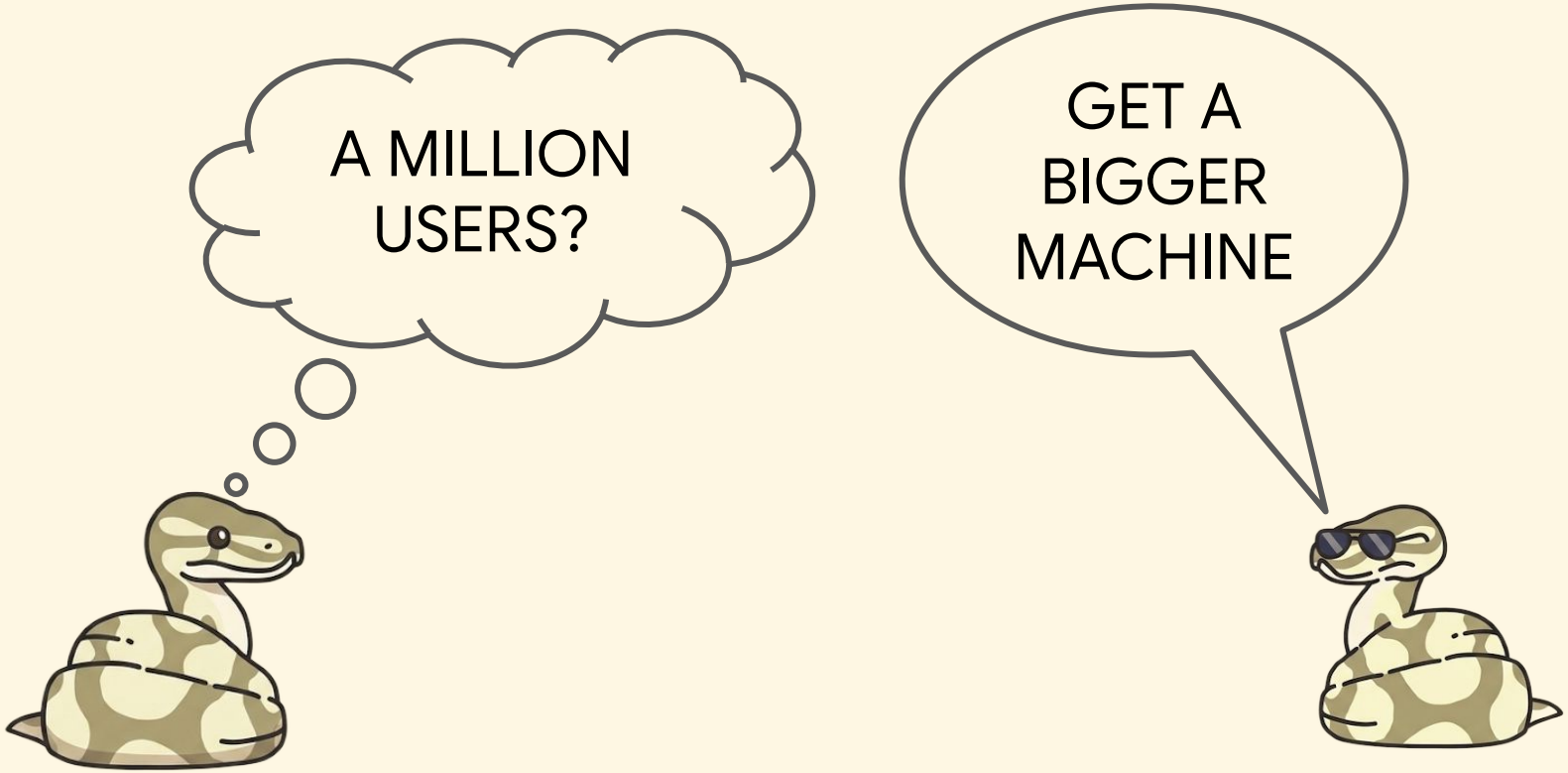
SINGLE NODE




```
make checkpoint STAGE=01

make lab STAGE=01

nwrite status pending

nread status

make incident STAGE=01
```




A MILLION
USERS?

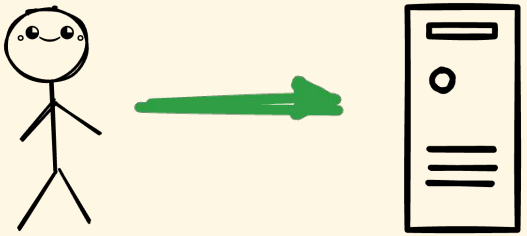

A MILLION USERS?
GET A BIGGER MACHINE

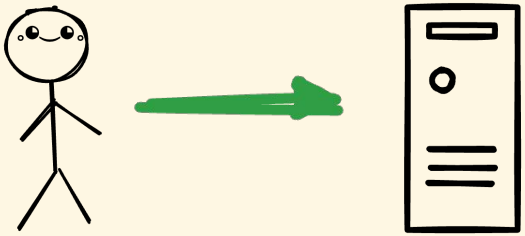

BIG AIN'T
ENOUGH!


BIG AIN'T ENOUGH!
GET MORE MACHINES

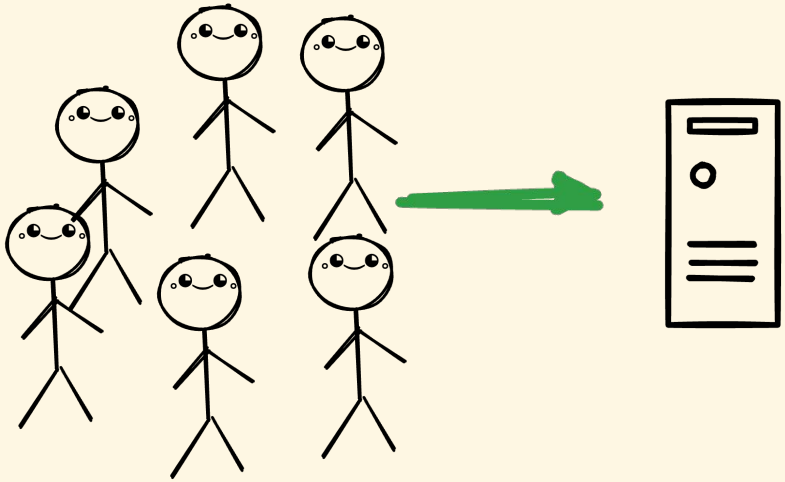

BIG AIN'T ENOUGH!
GET MORE MACHINES


BIG AIN'T ENOUGH!
GET MORE MACHINES



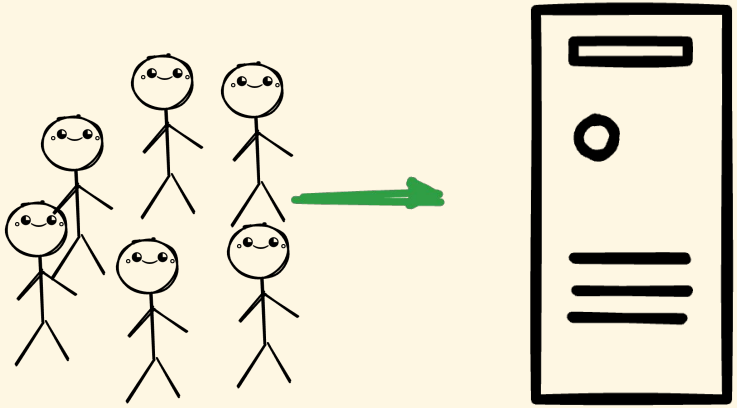


P95 = 200 ms



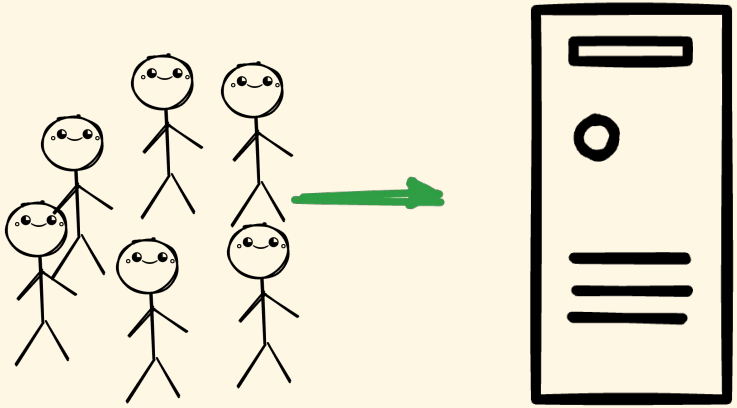
P95 = 800 ms

VERTICAL



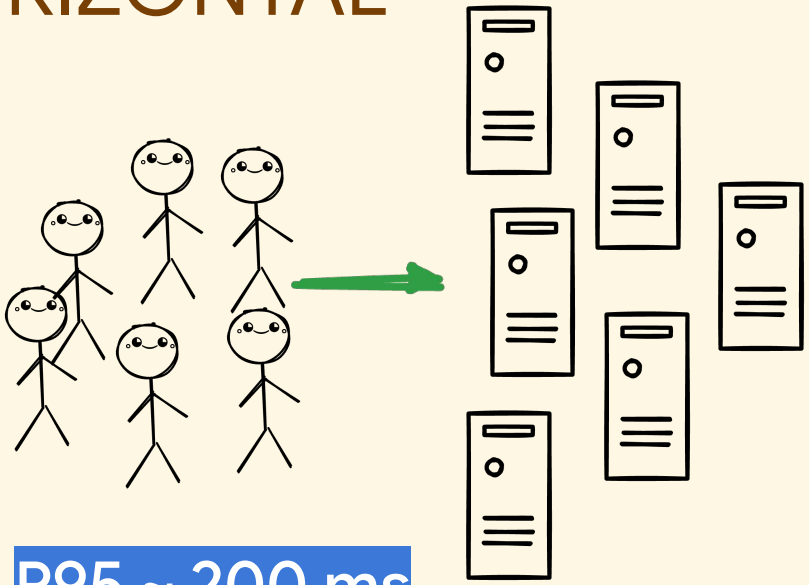
P95 ~ 200 ms

VERTICAL



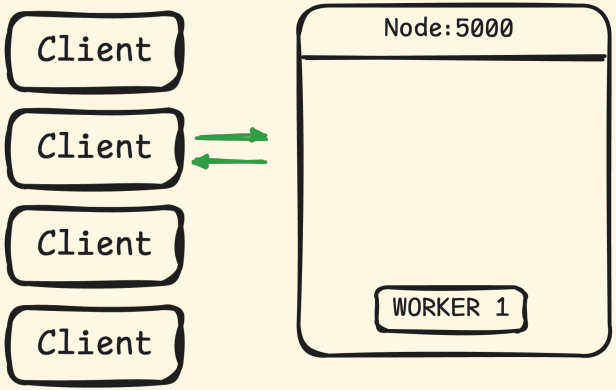
P95 ~ 200 ms

HORIZONTAL

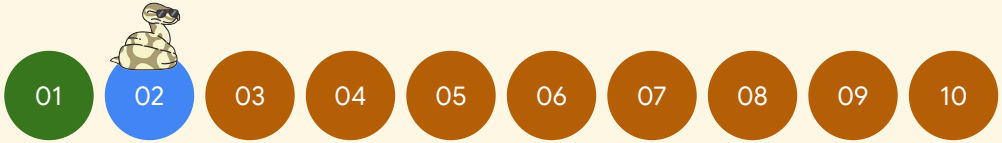


P95 ~ 200 ms

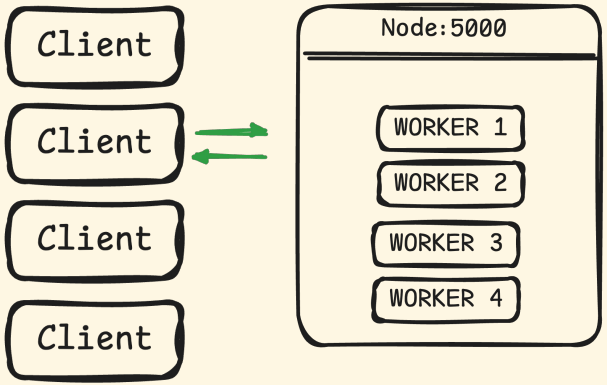
VERTICAL SCALING



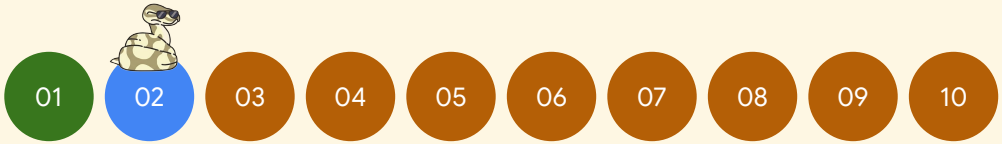

```
make checkpoint STAGE=02

make lab STAGE=02 WORKERS=1

make incident STAGE=02
```



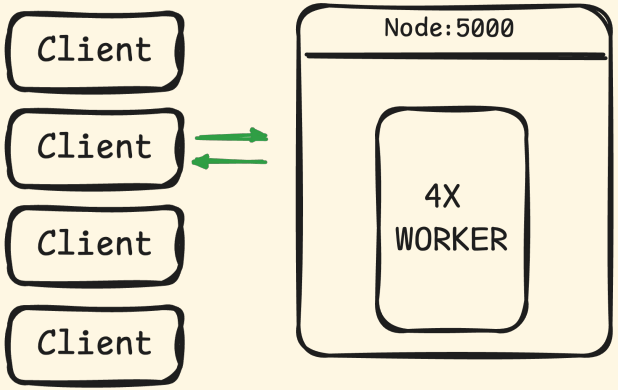
VERTICAL SCALING



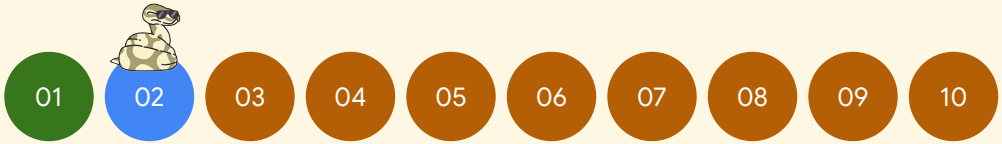

```
make checkpoint STAGE=02

make lab STAGE=02 WORKERS=4

make incident STAGE=02
```

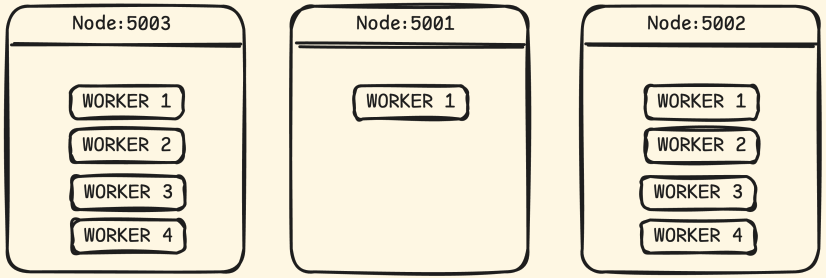


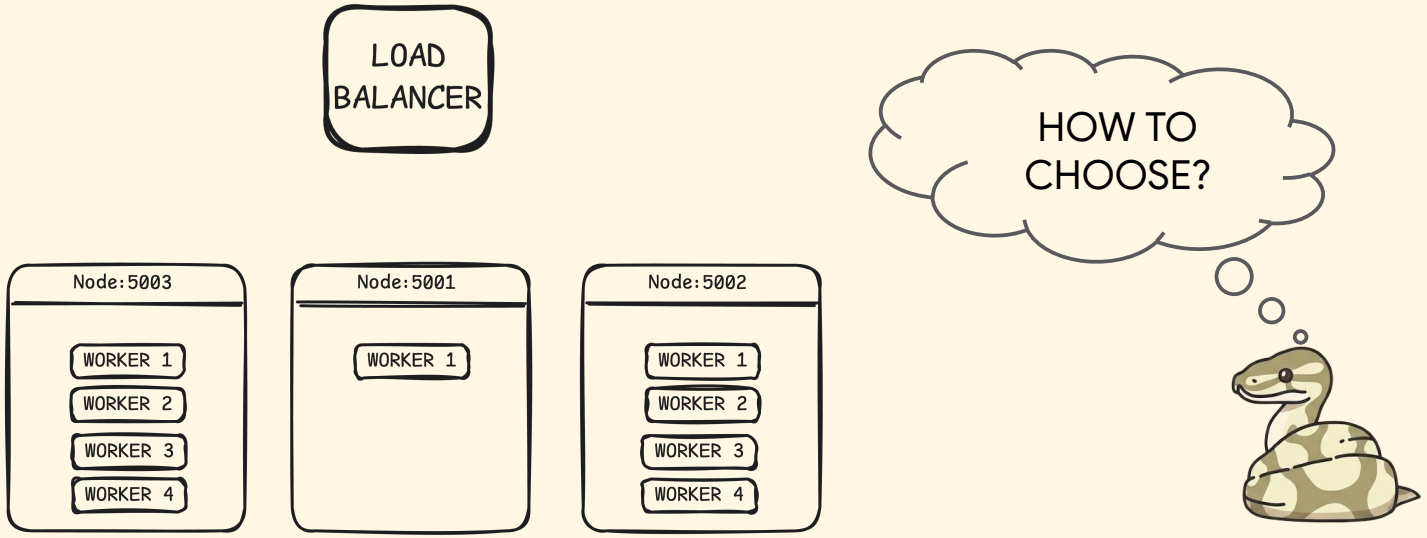
VERTICAL SCALING

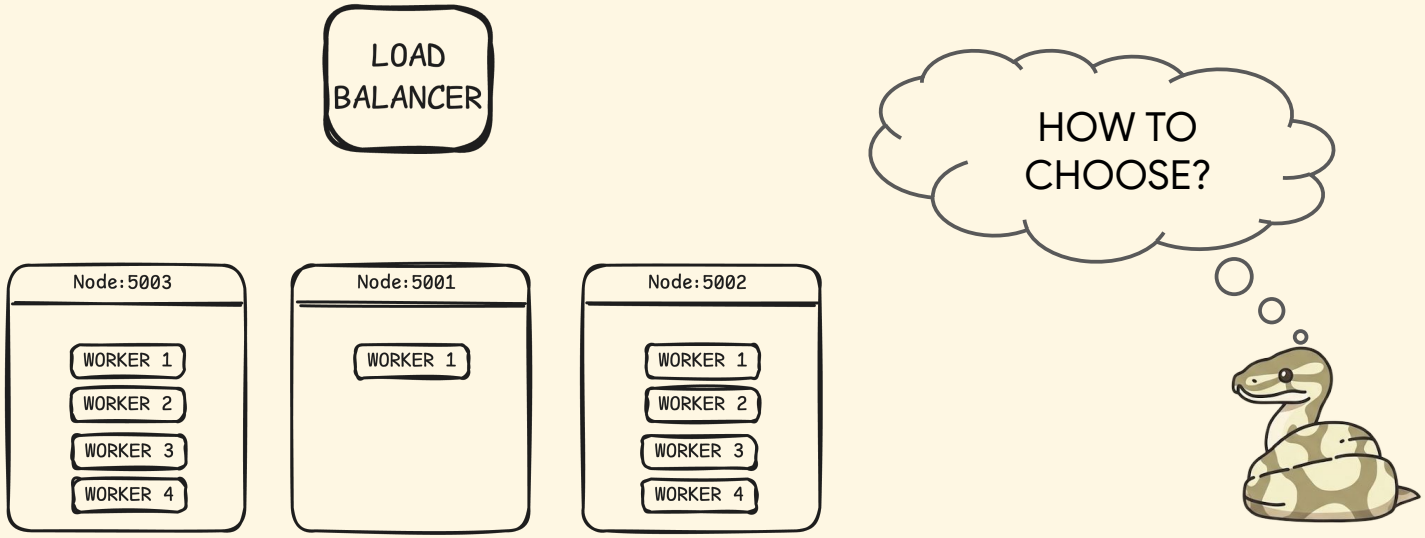


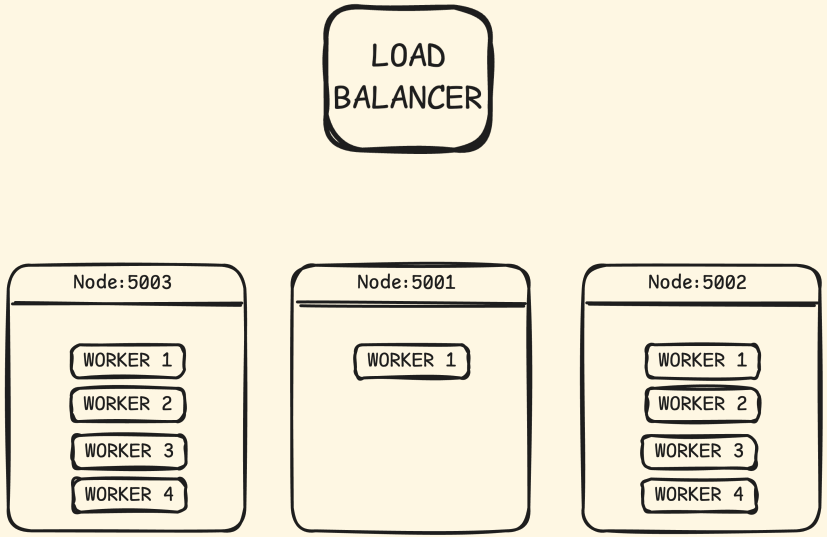

```
make checkpoint STAGE=02

make lab STAGE=02 WORKERS=4

make incident STAGE=02
```

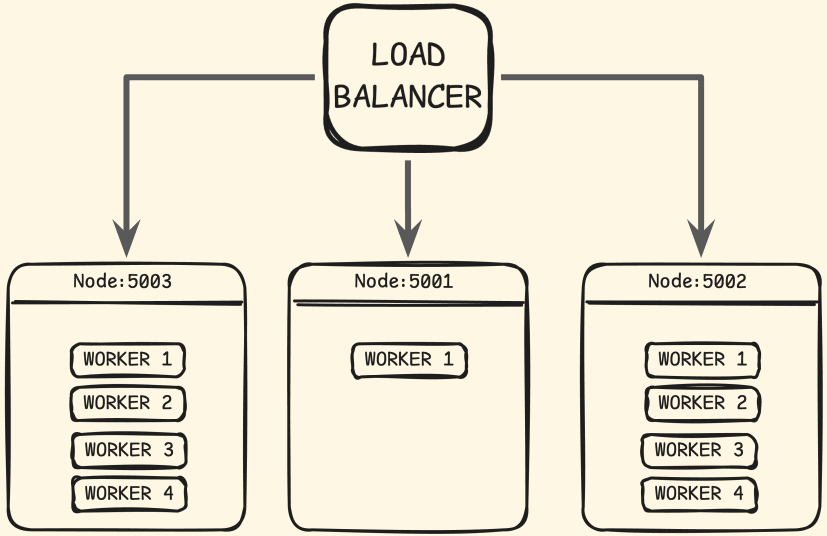


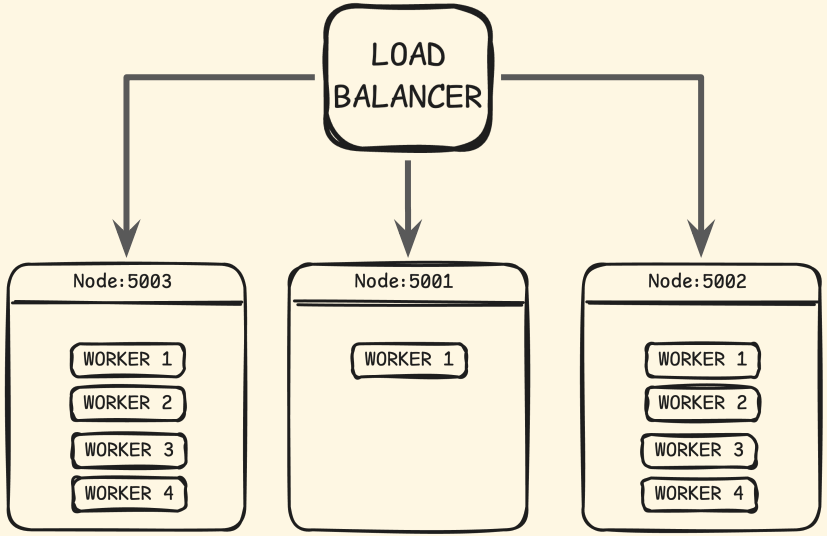

Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4
Node:5001
WORKER 1
Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4

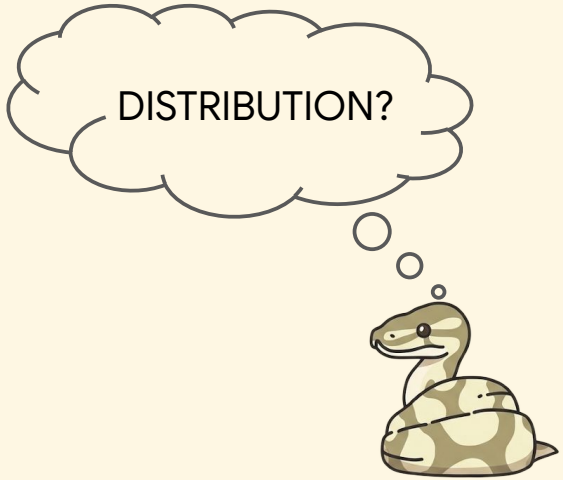

LOAD BALANCER
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4
Node:5001
WORKER 1
Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
HOW TO CHOOSE?


LOAD BALANCER
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4
Node:5001
WORKER 1
Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
HOW TO CHOOSE?

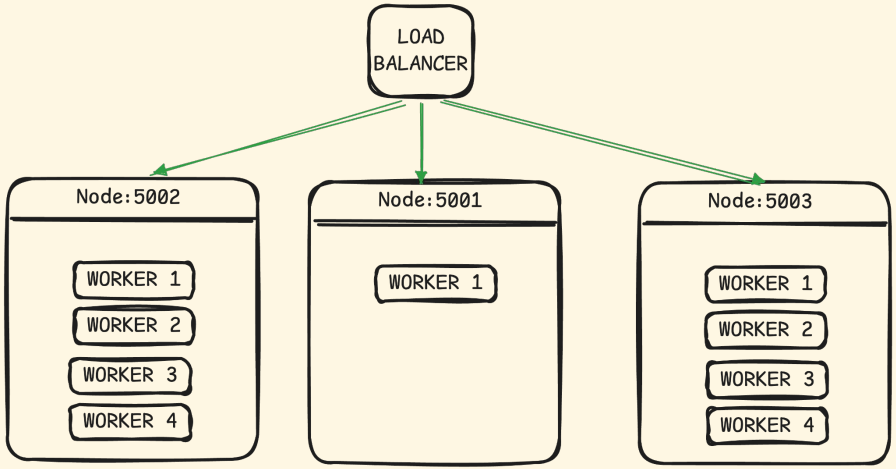

LOAD BALANCER
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4
Node:5001
WORKER 1
Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4


LOAD BALANCER
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4
Node:5001
WORKER 1
Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4

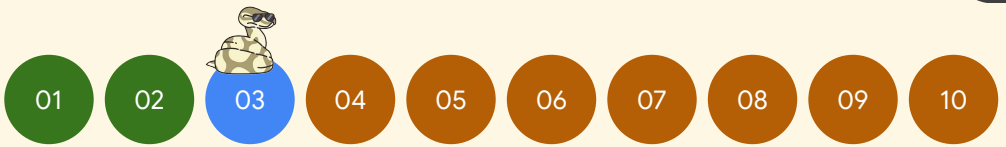

LOAD BALANCER
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4
Node:5001
WORKER 1
Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4


DISTRIBUTION?

HORIZONTAL SCALING ✏️



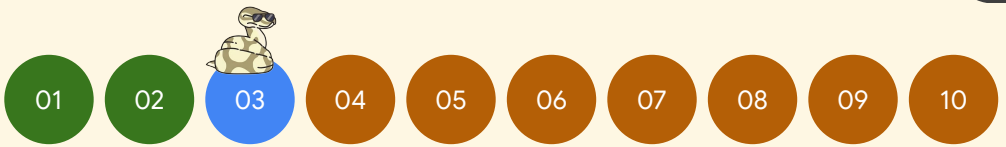

```
make todo STAGE=03

make checkpoint STAGE=03

make lab STAGE=03

nload round_robin 96 12

nload adaptive 96 12

make incident STAGE=03
```



HORIZONTAL SCALING ✏️

```python
class AdaptiveStrategy(LoadBalancerStrategy):
    """
    Adaptive Load Balancing Strategy.

    Prioritizes nodes with:
    1. Faster response times
    2. Fewer active requests

    Great for heterogeneous environments where nodes have different capacities.
    """
```

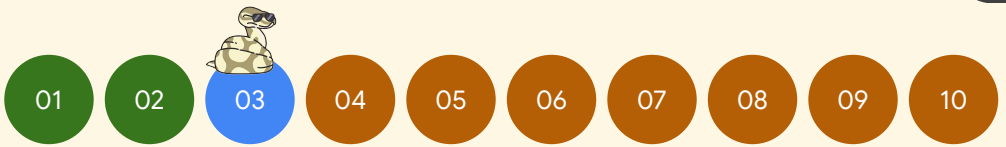
```
make todo STAGE=03

make checkpoint STAGE=03

make lab STAGE=03

nload round_robin 96 12

nload adaptive 96 12

make incident STAGE=03
```



HORIZONTAL SCALING ✏️

```python
def get_node(self, nodes: List[str]) -> str:
    if not nodes:
        raise ValueError("No nodes available")

    # TODO [STAGE 03]: adaptive selection — route to the BEST node.
    #   node_stats.get_score(node) returns a score per node (LOWER is better: it blends
    #   average latency and active-request count). Pick the node with the minimum scor
    #   and return its URL.
    raise NotImplementedError("STAGE 03: implement adaptive node selection")
```

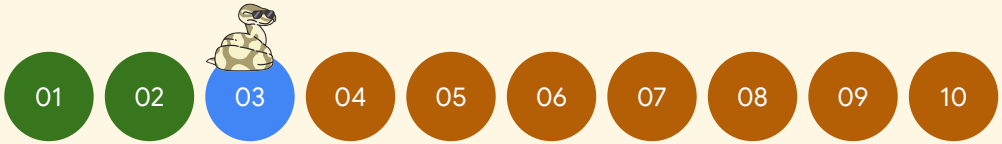
```
make todo STAGE=03

make checkpoint STAGE=03

make lab STAGE=03

nload round_robin 96 12

nload adaptive 96 12

make incident STAGE=03
```

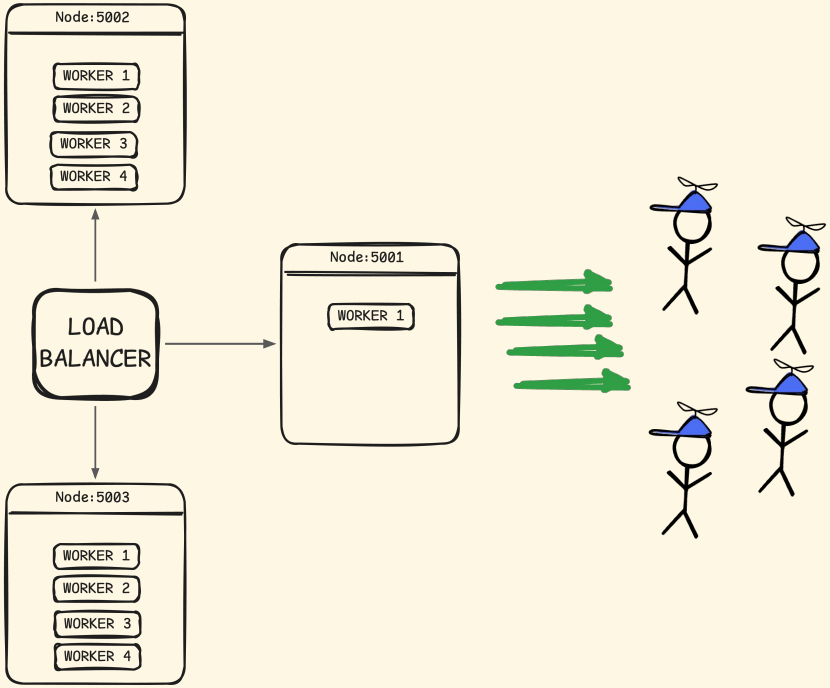


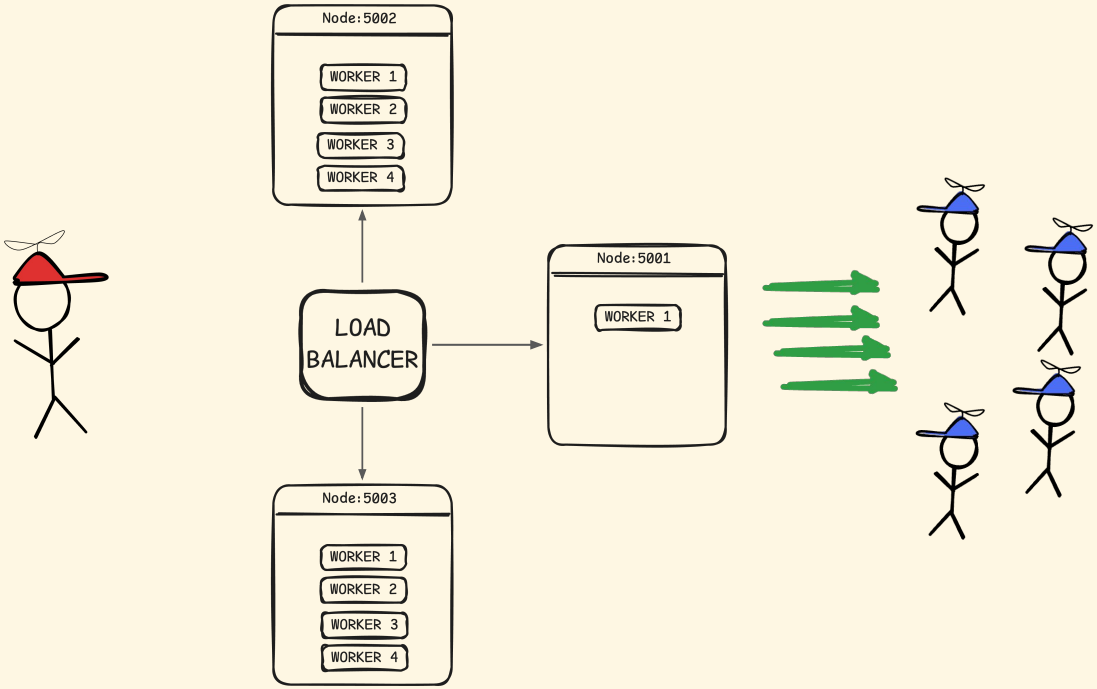
HORIZONTAL SCALING ✏️

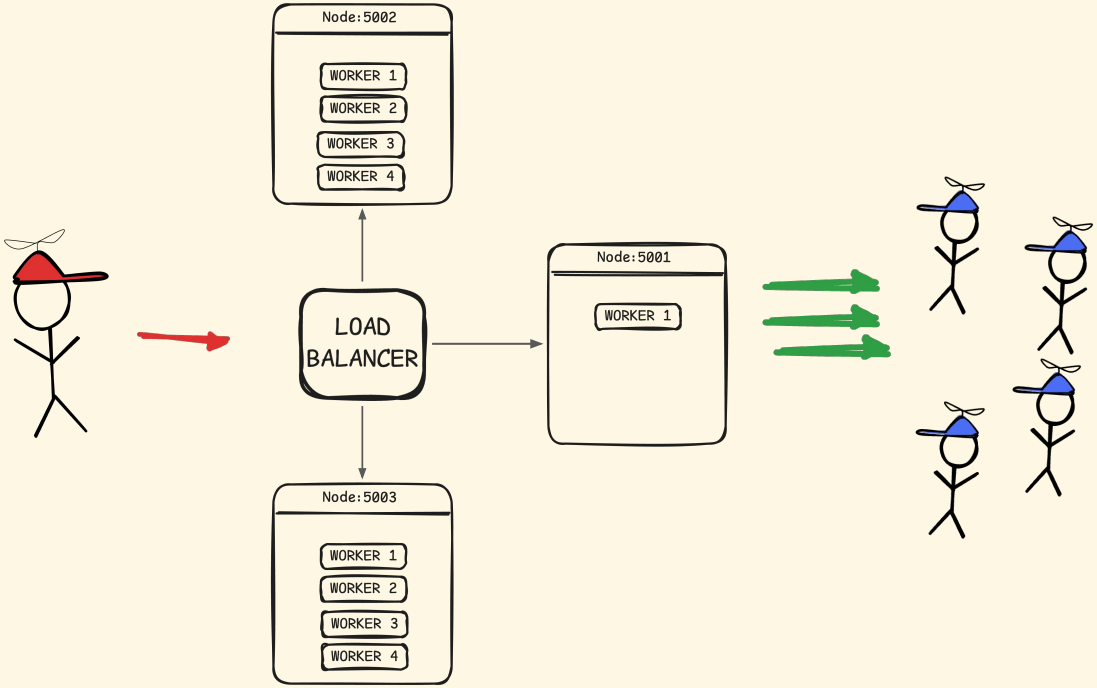
```python
def get_node(self, nodes: List[str]) -> str:
    if not nodes:
        raise ValueError("No nodes available")

    # Pick the lowest-score node (score blends latency + active requests; lower is better).
    # We only need the minimum, so this is a single O(n) pass — no full sort.
    return min(nodes, key=node_stats.get_score)
```

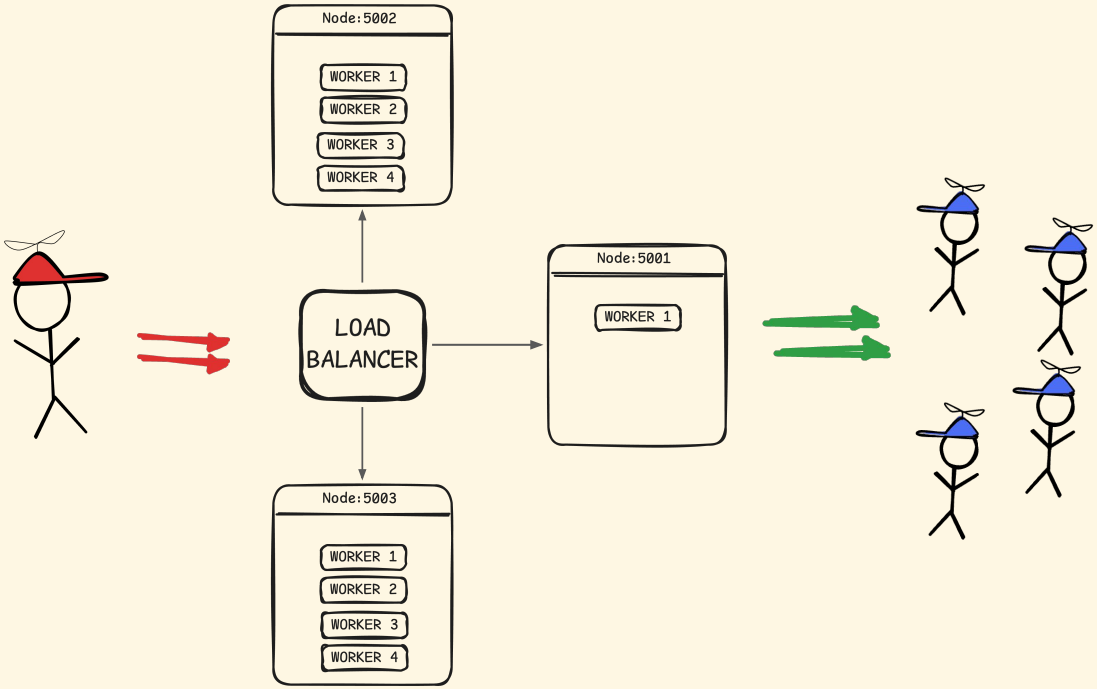
```
make todo STAGE=03

make checkpoint STAGE=03

make lab STAGE=03

nload round_robin 96 12

nload adaptive 96 12

make incident STAGE=03
```

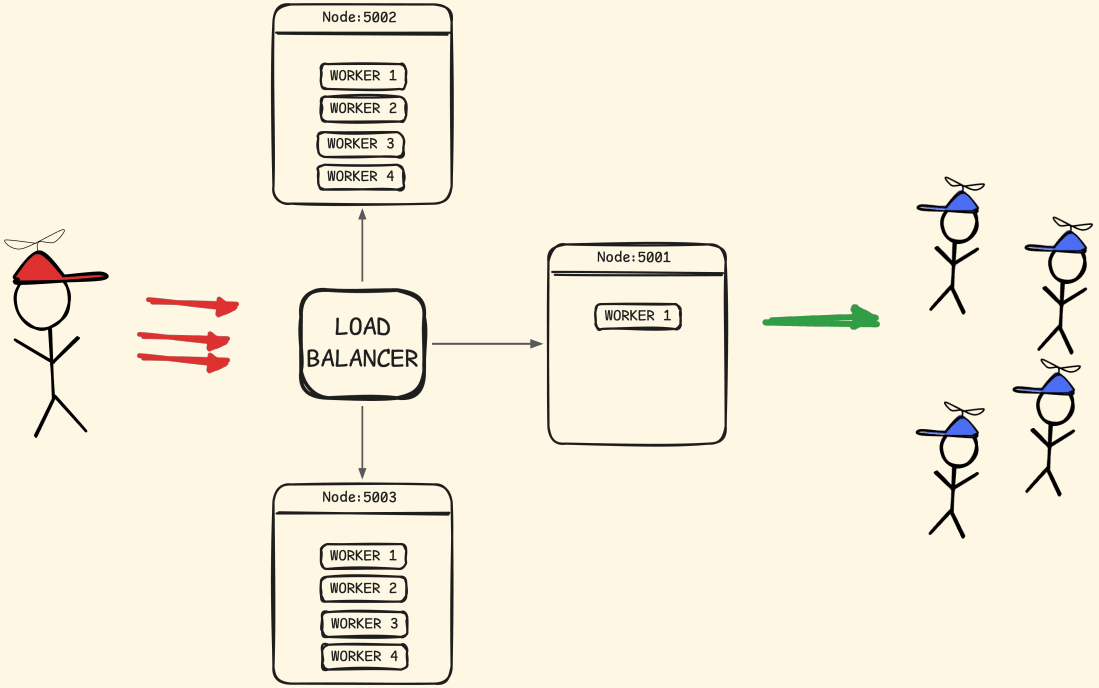


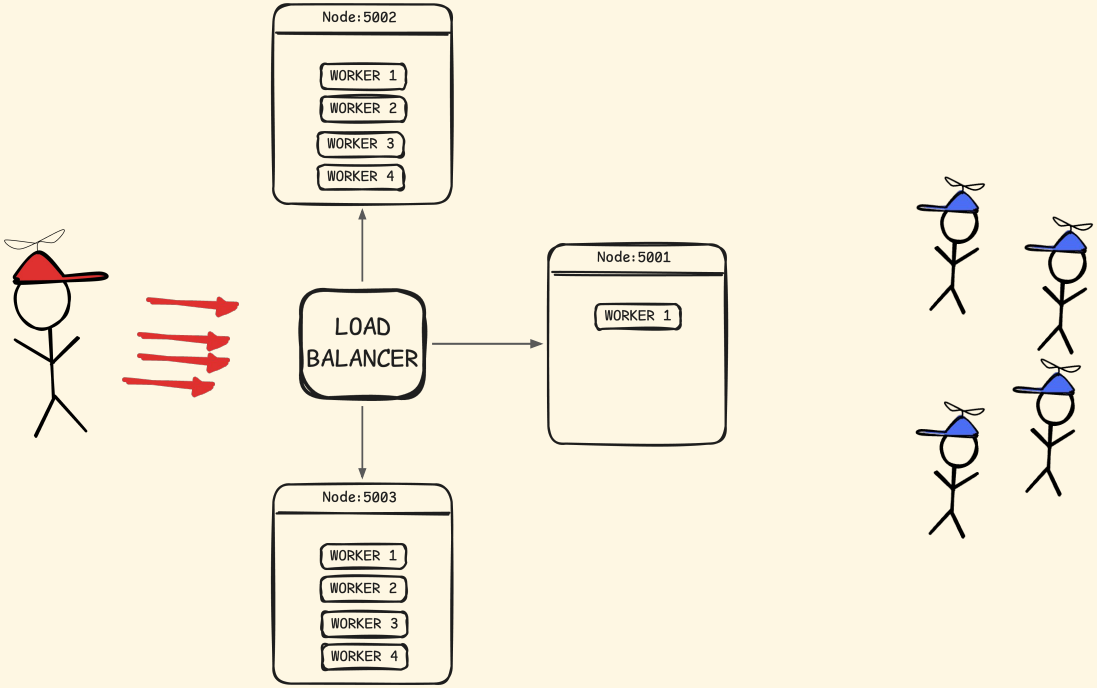

Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
LOAD BALANCER
Node:5001
WORKER 1
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4


Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
LOAD BALANCER
Node:5001
WORKER 1
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4

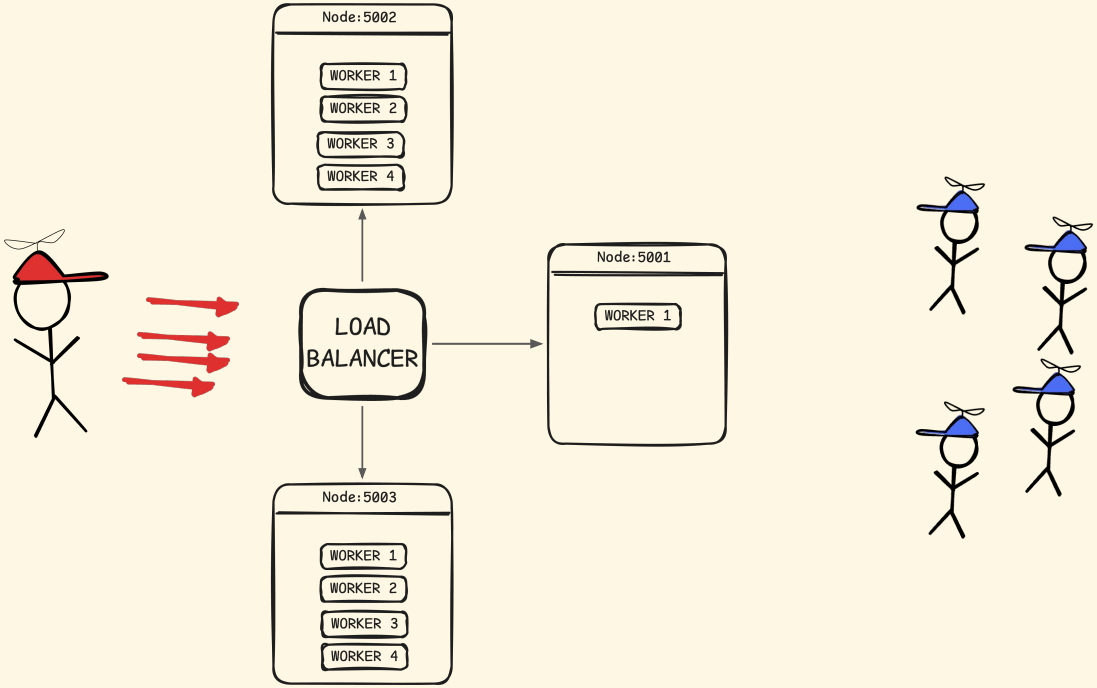

Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
LOAD BALANCER
Node:5001
WORKER 1
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4


Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
LOAD BALANCER
Node:5001
WORKER 1
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4


Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
LOAD BALANCER
Node:5001
WORKER 1
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4

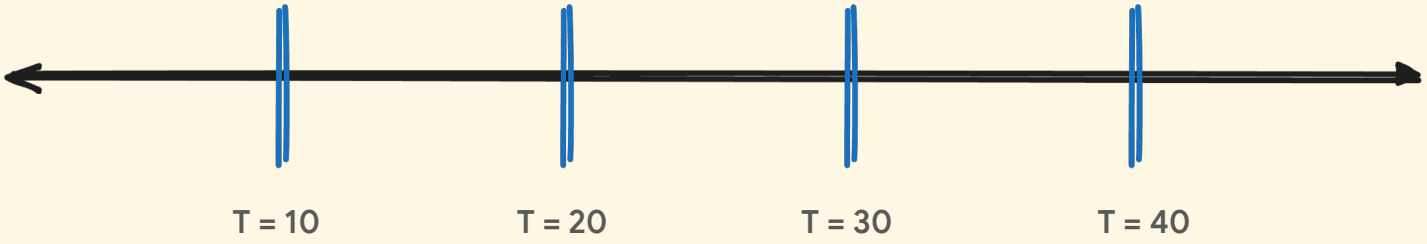

Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
LOAD BALANCER
Node:5001
WORKER 1
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4

DDoS

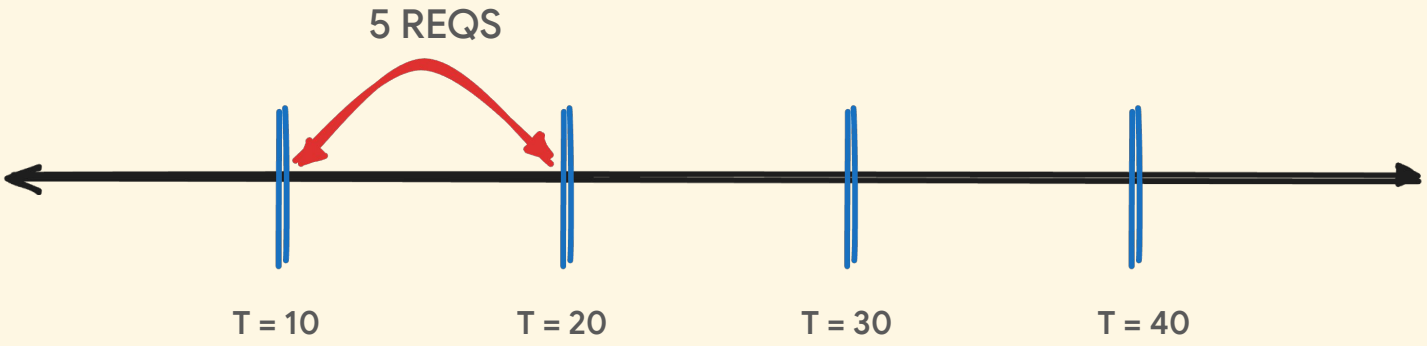

Node:5002
WORKER 1
WORKER 2
WORKER 3
WORKER 4
LOAD BALANCER
Node:5001
WORKER 1
Node:5003
WORKER 1
WORKER 2
WORKER 3
WORKER 4

FIXED WINDOW RATE LIMITING

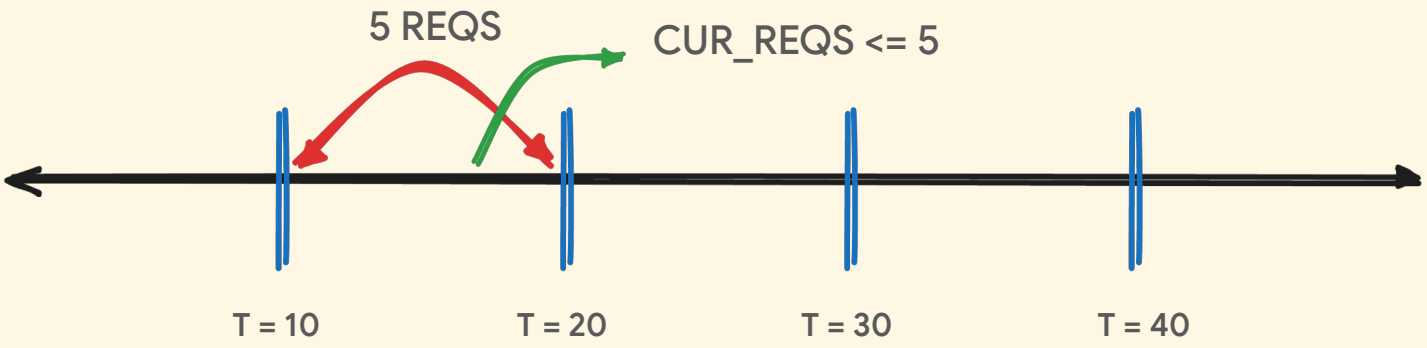
FIXED WINDOW RATE LIMITING



FIXED WINDOW RATE LIMITING



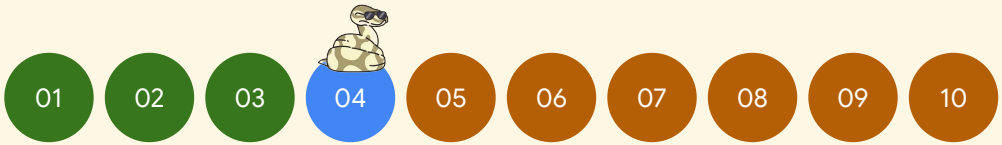
FIXED WINDOW RATE LIMITING



RATE LIMITING ✏️

```python
def is_allowed(self, client_id: str) -> Tuple[bool, dict]:
    now = time.time()
    bucket = self.buckets[client_id]
```

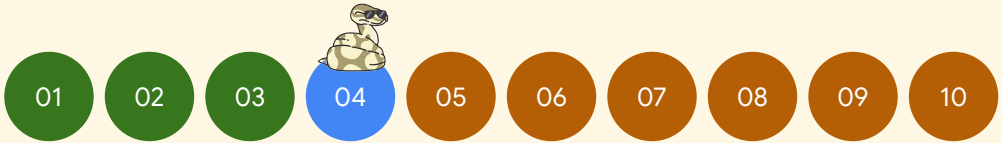
```
make todo STAGE=04

make checkpoint STAGE=04

make lab STAGE=04

make incident STAGE=04
```

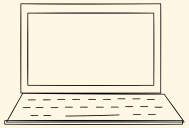


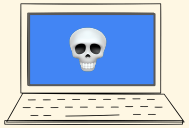
RATE LIMITING ✏️

```python
def is_allowed(self, client_id: str) -> Tuple[bool, dict]:
    now = time.time()
    bucket = self.buckets[client_id]

    # Reset window if it has expired
    if now - bucket["window_start"] >= self.window_seconds:
        bucket["window_start"] = now
        bucket["count"] = 0

    # Allow if under the limit, else reject
    allowed = bucket["count"] < self.max_requests
    if allowed:
        bucket["count"] += 1
```

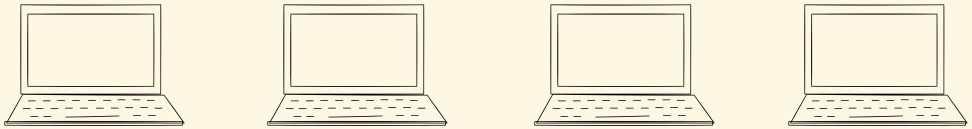
```
make todo STAGE=04

make checkpoint STAGE=04

make lab STAGE=04

make incident STAGE=04
```

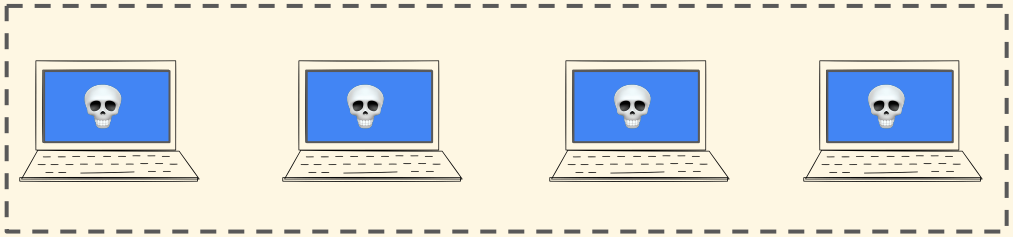






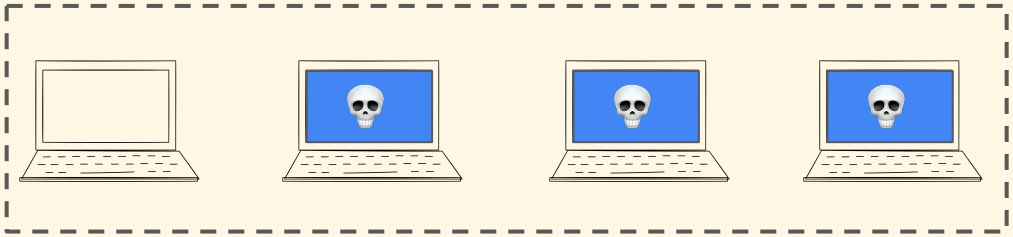
F = Fault
P(F) = 0.000001



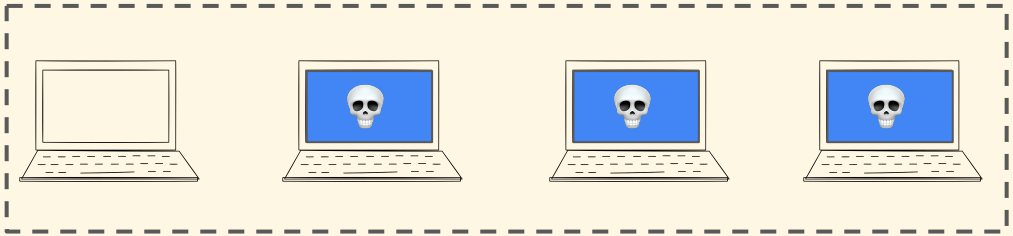


F = Fault

P(F) = 0.000001 x 0.000001 x 0.000001 x 0.000001 = 1/10^24 SMALL



1 - P(F) = 1 - 1/10^24 ~ 1



1 - P(F) = 1 - 1/10^24 ~ 1

SMALL IS LARGE!

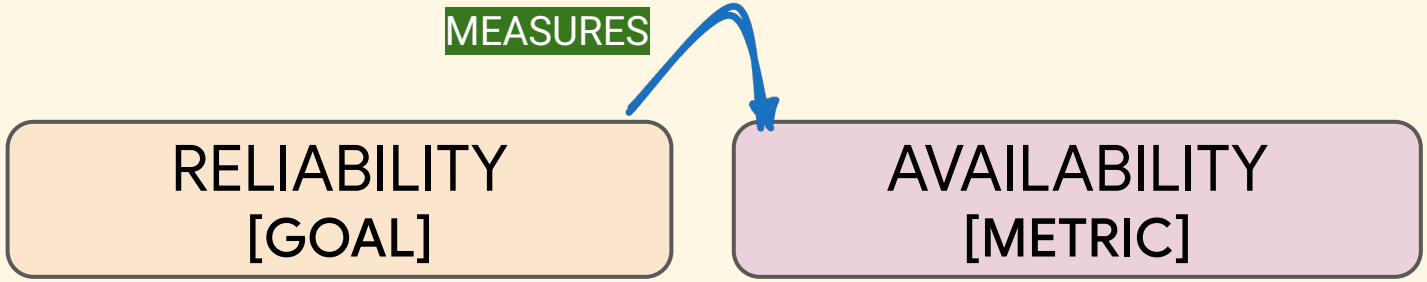
FAULTS ARE REAL

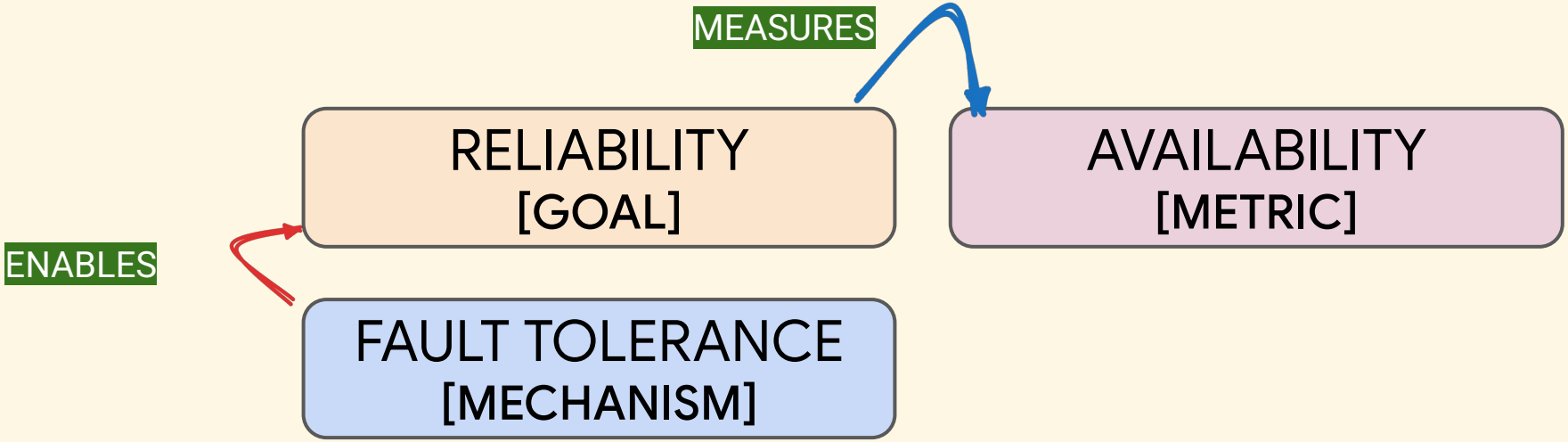
FAULTS ARE REAL

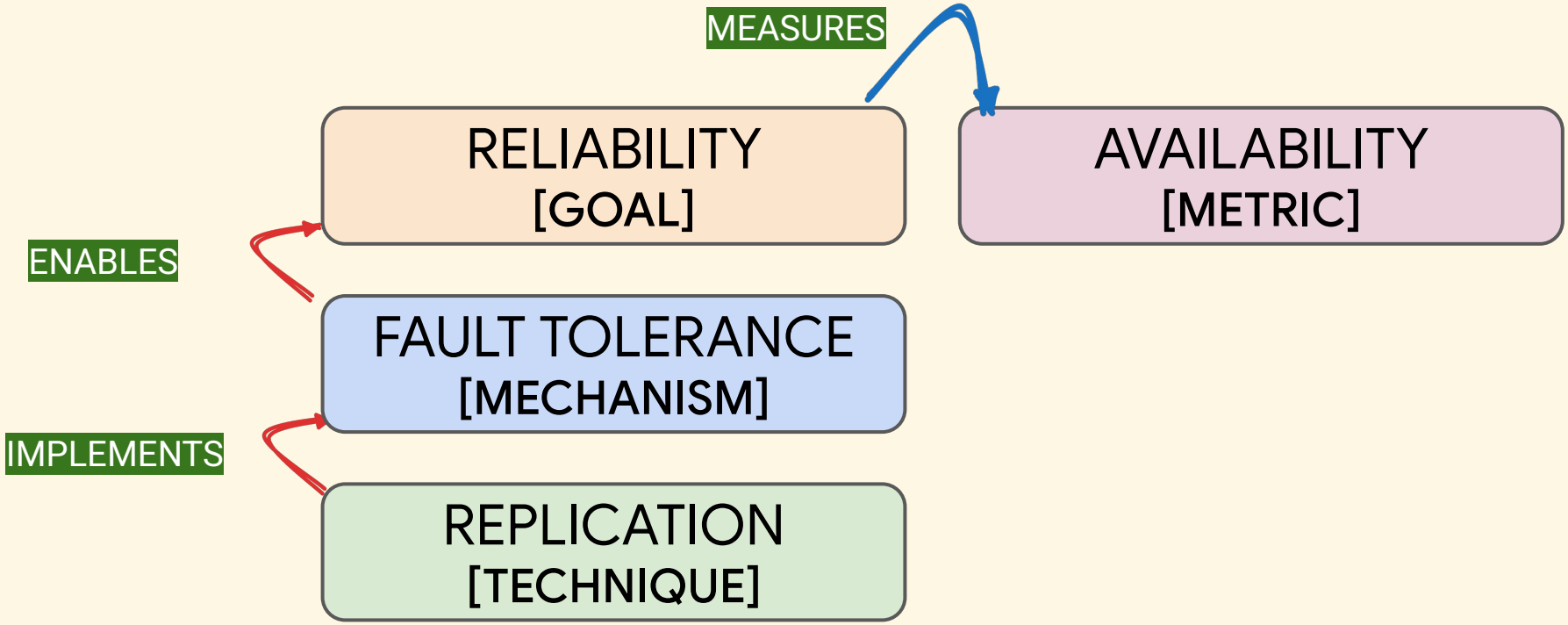
FAILURE IS A CHOICE!

Availability	Downtime / Year	Downtime / Month	Downtime / Week	Downtime / Day
99.999%	5.256 Minutes	0.438 Minutes	0.101 Minutes	0.014 Minutes
99.995%	26.28 Minutes	2.19 Minutes	0.505 Minutes	0.072 Minutes
99.990%	52.56 Minutes	4.38 Minutes	1.011 Minutes	0.144 Minutes
99.950%	4.38 Hours	21.9 Minutes	5.054 Minutes	0.72 Minutes
99.900%	8.76 Hours	43.8 Minutes	10.108 Minutes	1.44 Minutes
99.500%	43.8 Hours	3.65 Hours	50.538 Minutes	7.2 Minutes
99.250%	65.7 Hours	5.475 Hours	75.808 Minutes	10.8 Minutes
99.000%	87.6 Hours	7.3 Hours	101.077 Minutes	14.4 Minutes

RELIABILITY
[GOAL]


MEASURES
RELIABILITY
[GOAL]
AVAILABILITY
[METRIC]


MEASURES
RELIABILITY
[GOAL]
AVAILABILITY
[METRIC]
ENABLES
FAULT TOLERANCE
[MECHANISM]


MEASURES
RELIABILITY
[GOAL]
AVAILABILITY
[METRIC]
ENABLES
FAULT TOLERANCE
[MECHANISM]
IMPLEMENTS
REPLICATION
[TECHNIQUE]

MEASURES

RELIABILITY
[GOAL]

AVAILABILITY
[METRIC]

ENABLES

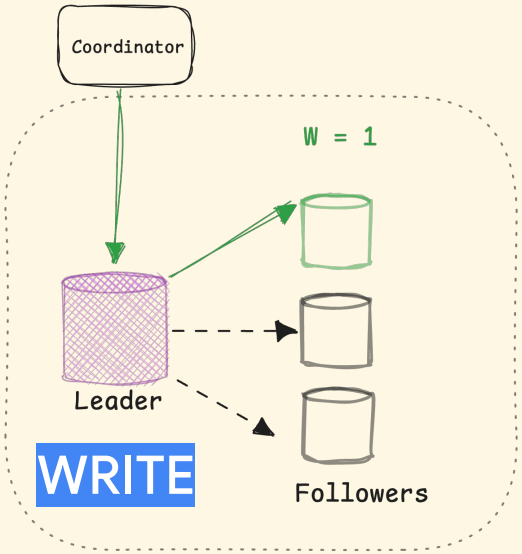
FAULT TOLERANCE
[MECHANISM]

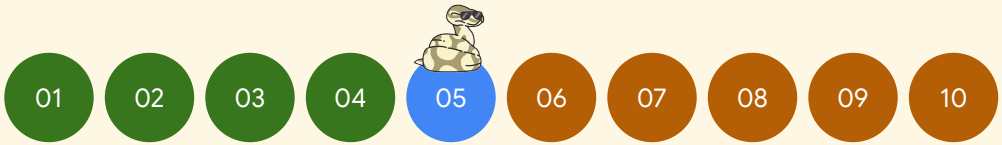
SCALING
STATEFUL
SYSTEMS!

IMPLEMENTS

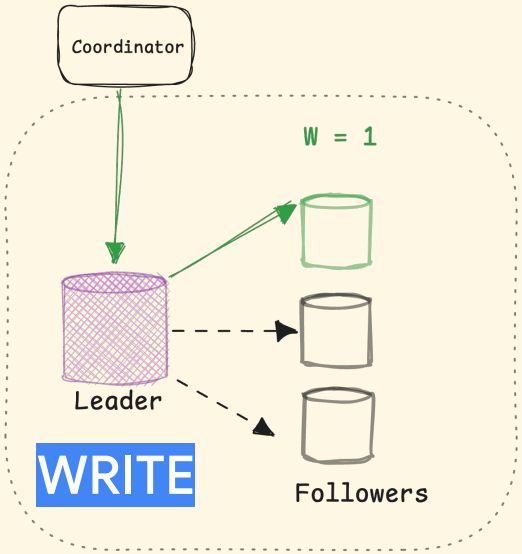
REPLICATION
[TECHNIQUE]

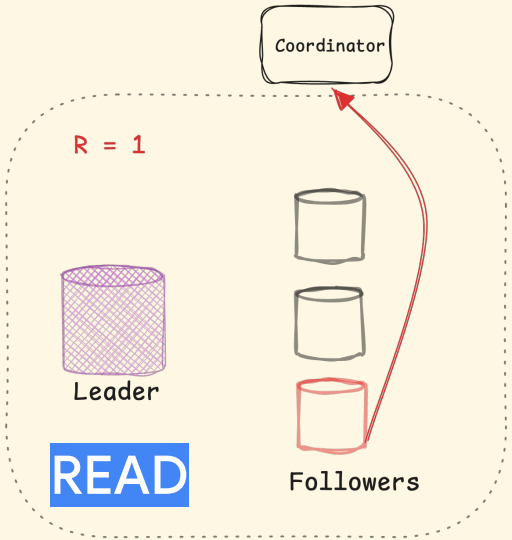
WAIT FOR ONE ✏️

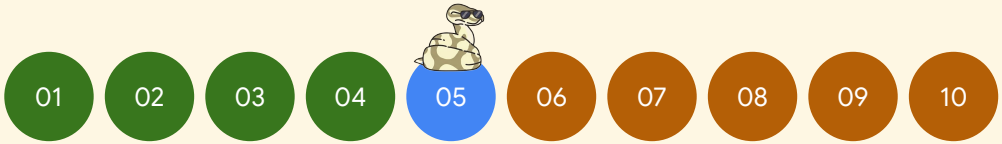

Coordinator
W = 1
Leader
WRITE
Followers


01
02
03
04
05
06
07
08
09
10

WAIT FOR ONE ✏️


Coordinator
W = 1
Leader
WRITE
Followers

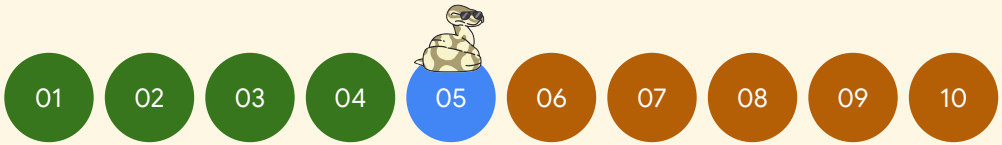

Coordinator
R = 1
Leader
READ
Followers


01
02
03
04
05
06
07
08
09
10

WAIT FOR ONE ✏️

```python
def replicate_sync(sync_followers: List[str], key: str, value: str, version: int) -> dict:
    """
    Replicate to sync followers synchronously (wait for all acks).
    Uses short SYNC_DELAY.
    Returns dict with ack count and details.
    """
```

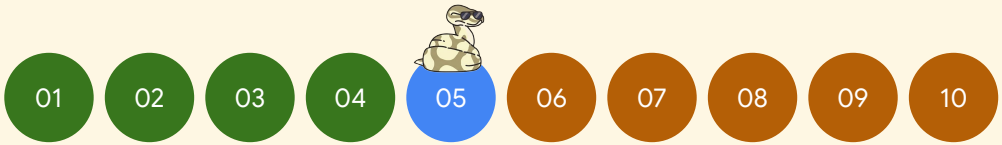
```
make todo STAGE=04

make checkpoint STAGE=04

make lab STAGE=04

make incident STAGE=04
```



WAIT FOR ONE ✏️

```python
def replicate_async(async_followers: List[str], key: str, value: str, version: int):
    """
    Replicate to async followers in background (don't wait).
    Uses long ASYNC_DELAY.
    """
```

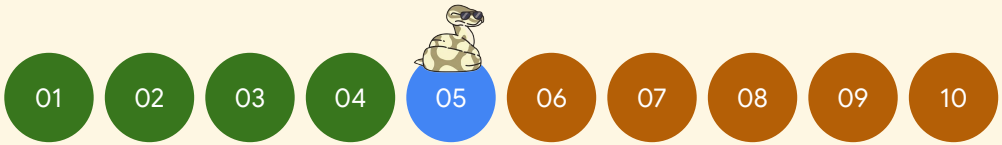
```
make todo STAGE=04

make checkpoint STAGE=04

make lab STAGE=04

make incident STAGE=04
```



WAIT FOR ONE ✏️

```
try:
    # TODO [STAGE 05]: replication itself — send the write to the follower over HTTP.
    #   resp = requests.post(
    #       f"{follower_url}/replicate",
    #       json={"key": key, "value": value, "version": version, "source": NODE_ID},
    #       timeout=10,
    #   )
    # Replace the line below with that POST. Everything after it is done for you.
    raise NotImplementedError("STAGE 05: POST the write to the follower's /replicate")
```

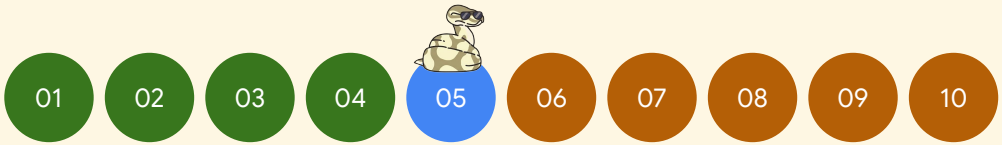
```
make todo STAGE=04

make checkpoint STAGE=04

make lab STAGE=04

make incident STAGE=04
```



WAIT FOR ONE ✏️

```
try:
    resp = requests.post(
        f"{follower_url}/replicate",
        json={"key": key, "value": value, "version": version, "source": NODE_ID},
        timeout=10
    )
```

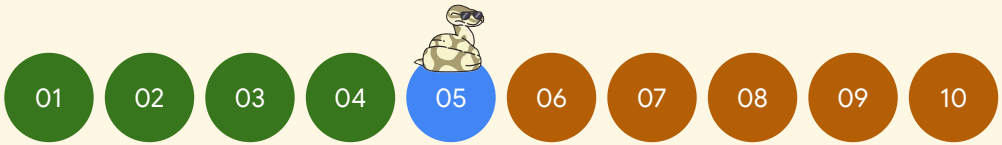
```
make todo STAGE=04

make checkpoint STAGE=04

make lab STAGE=04

make incident STAGE=04
```



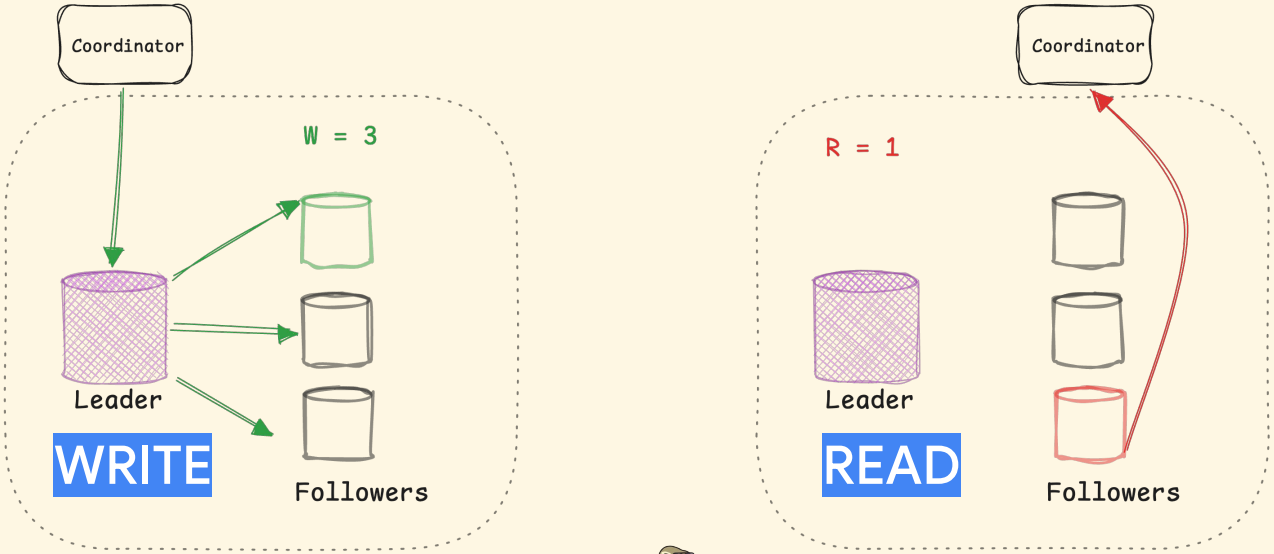
WAIT FOR ONE ✏️

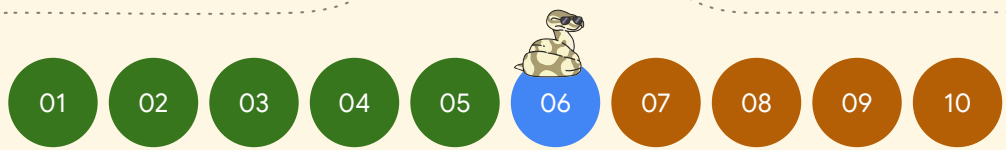
```python
try:
    resp = requests.post(
        f"{follower_url}/replicate",
        json={"key": key, "value": value, "version": version, "source": NODE_ID},
        timeout=10
    )
```

```
kvwrite order pending

kvwrite order done

kvread order
```



WAIT FOR ALL


Coordinator
W = 3
Leader
WRITE
Followers
Coordinator
R = 1
Leader
READ
Followers


01
02
03
04
05
06
07
08
09
10

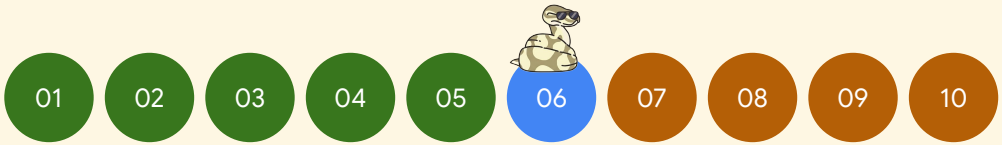
WAIT FOR ALL

W = 1

R = 1

STALE

```
make checkpoint STAGE=06
make lab STAGE=06
kvwrite order paid
kvread order
kvkill 1
kvwrite order delivered
kvread order
```

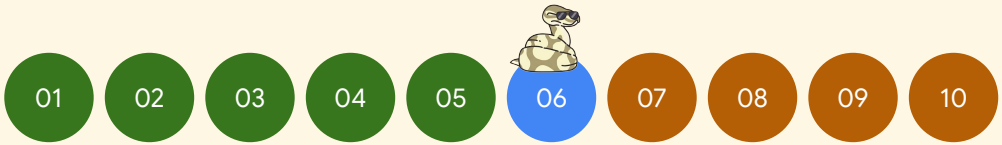


WAIT FOR ALL

W = 1 R = 1 STALE

W = 3 R = 1 FRAGILE

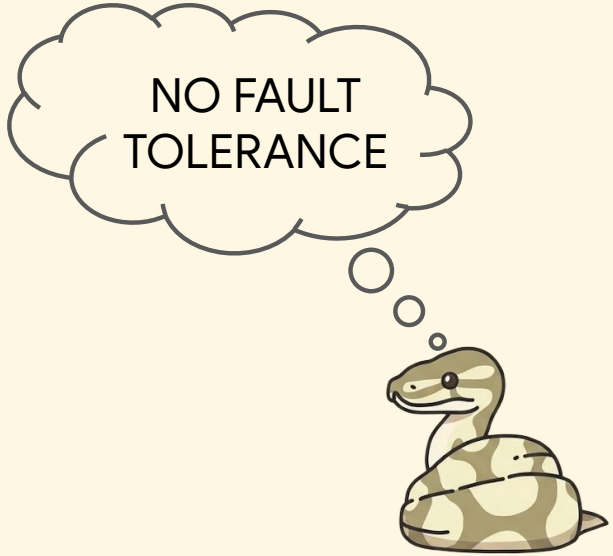
```
make checkpoint STAGE=06
make lab STAGE=06
kvwrite order paid
kvread order
kvkill 1
kvwrite order delivered
kvread order
```

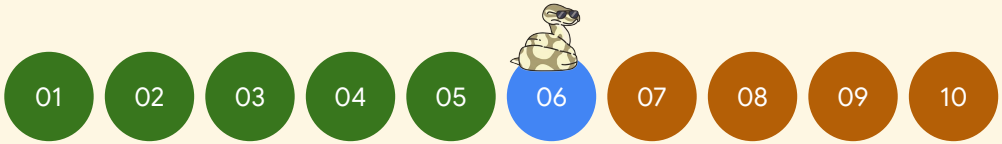


WAIT FOR ALL

W = 1 R = 1 STALE

W = 3 R = 1 FRAGILE

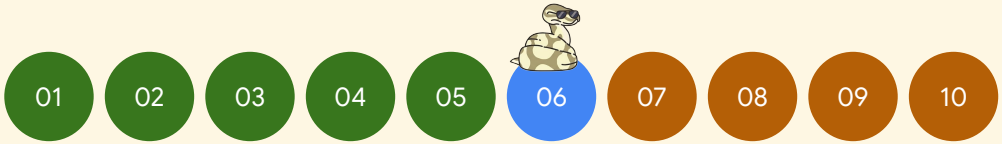




WAIT FOR ALL

W = 1 R = 1 STALE

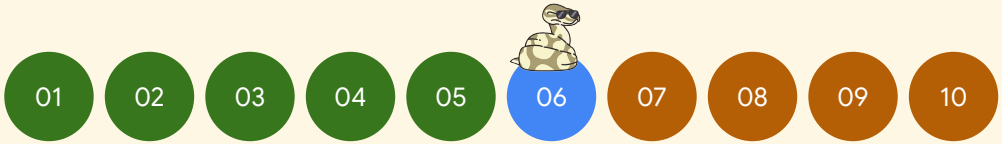
W = 3 R = 1 FRAGILE

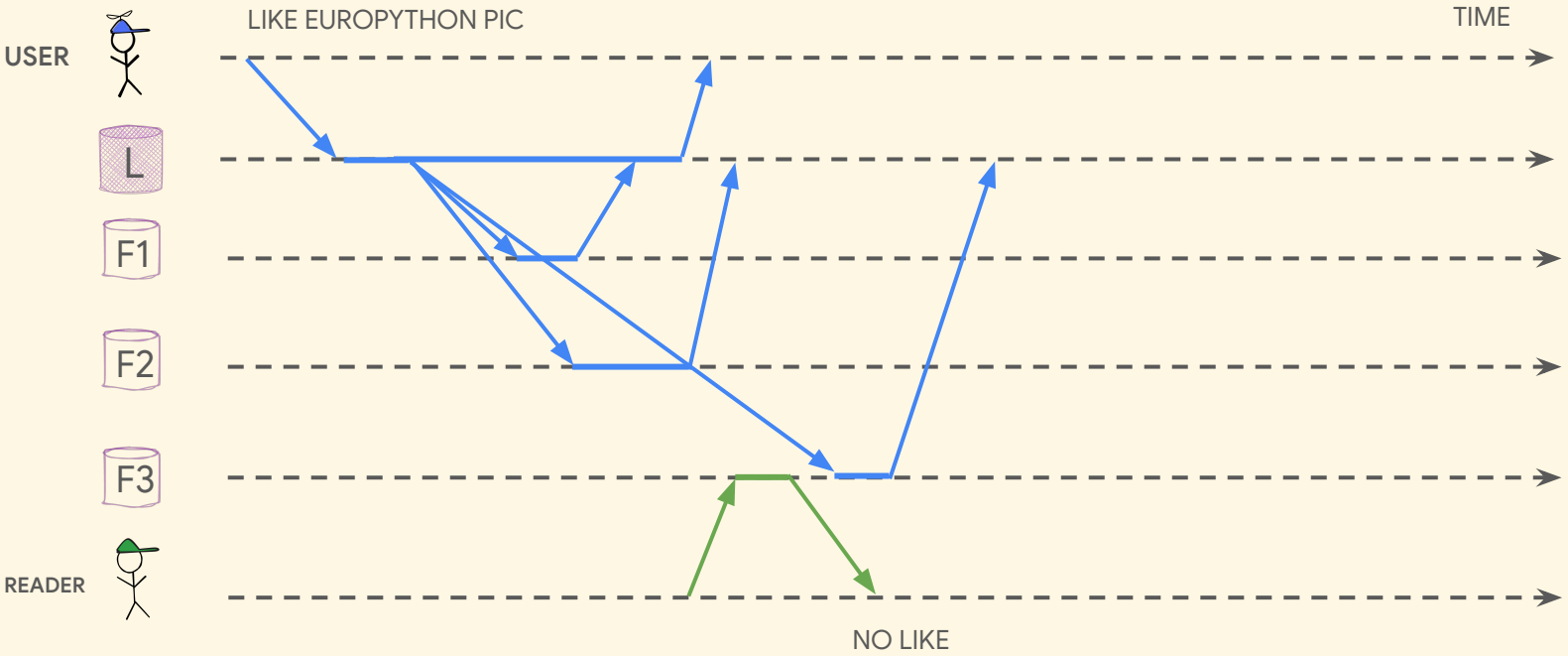


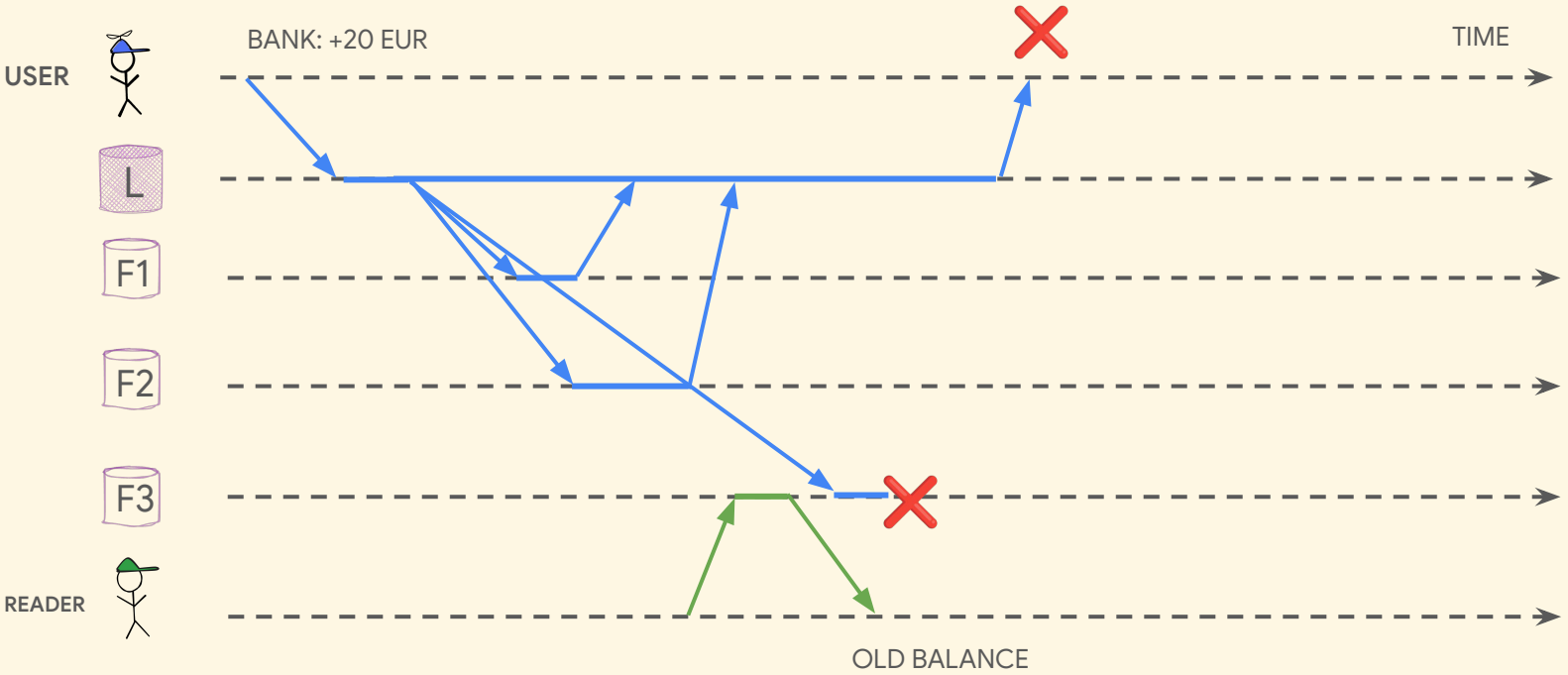
WAIT FOR ALL

W = 1 R = 1 STALE

W = 3 R = 1 FRAGILE




LIKE EUROPYTHON PIC
TIME
USER
L
F1
F2
F3
READER
NO LIKE


BANK: +20 EUR
TIME
USER
L
F1
F2
F3
READER
OLD BALANCE

CONSISTENCY = NO STALE DATA

AVAILABILITY = ALWAYS ANSWER

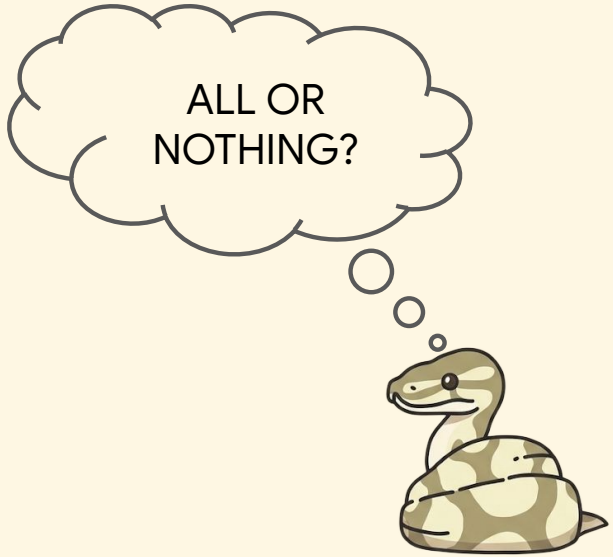
CAP THEOREM

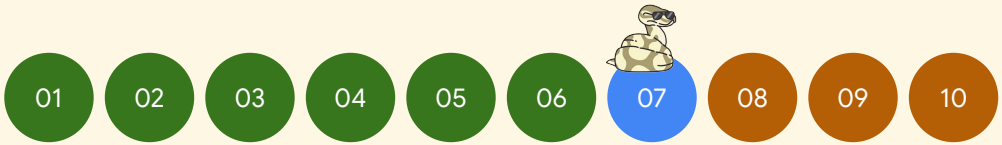
AVAILABILITY OR CONSISTENCY

THE MAJORITY

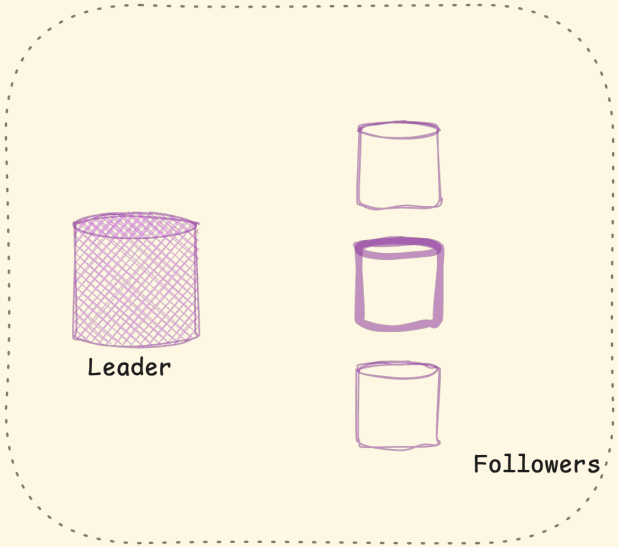
W = 1 R = 1 STALE

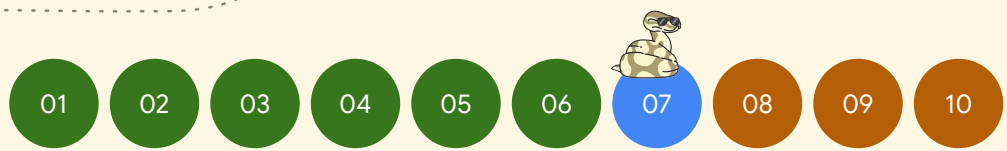
W = 3 R = 1 FRAGILE



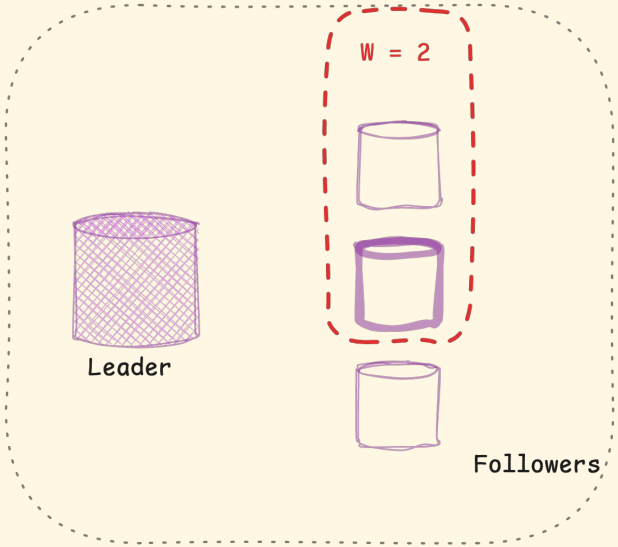


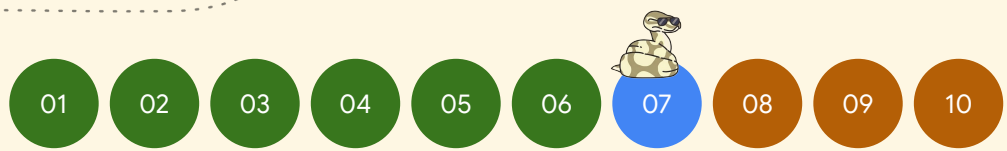
THE MAJORITY



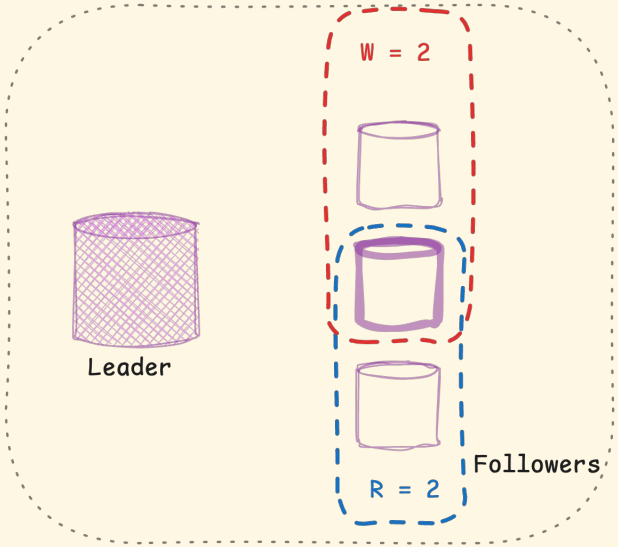


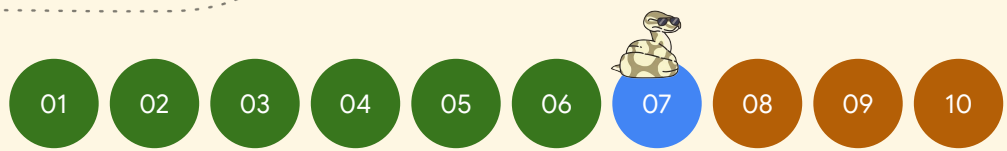
THE MAJORITY



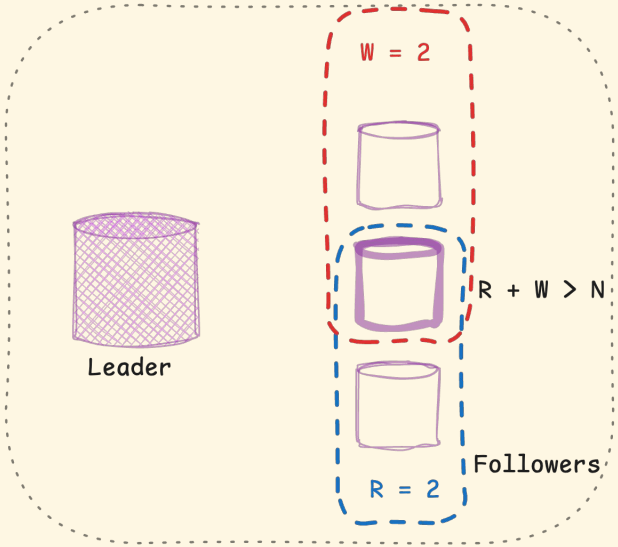


THE MAJORITY

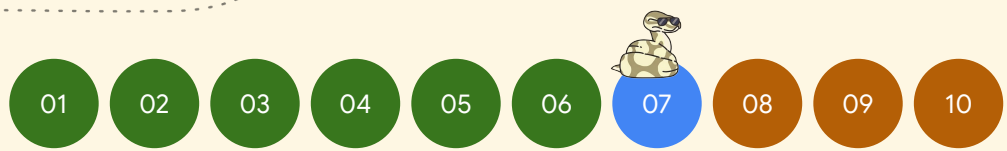




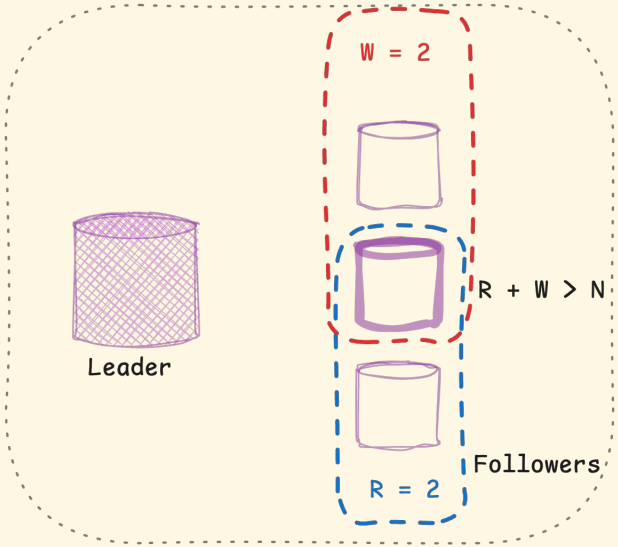
THE MAJORITY



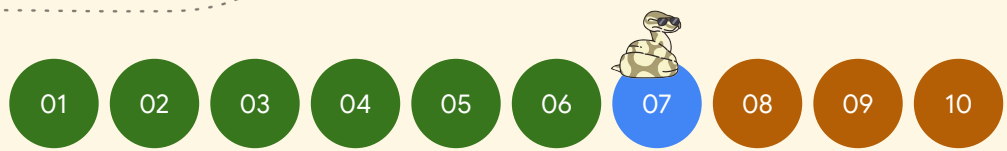
WRITE TO ENOUGH!
READ TO ENOUGH!



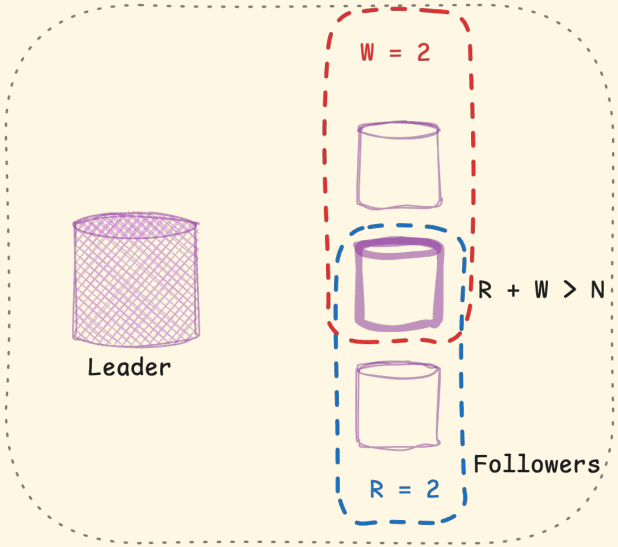
THE MAJORITY



QUORUM IS INTERSECTION

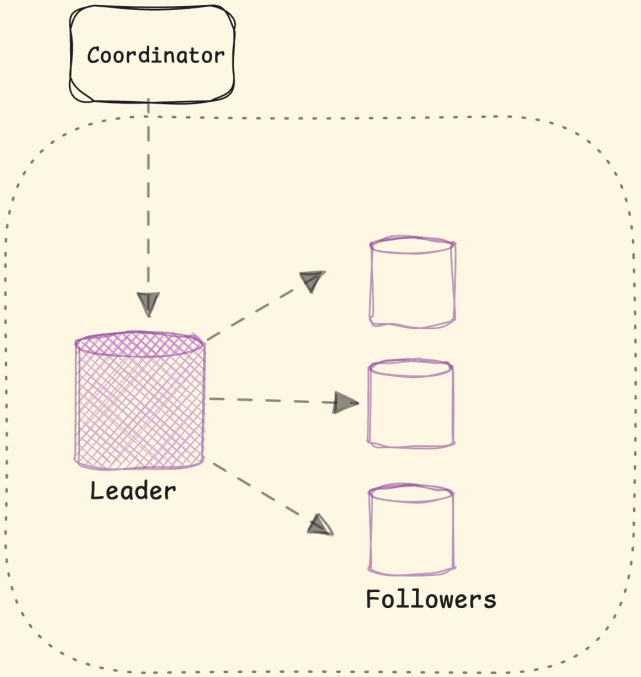


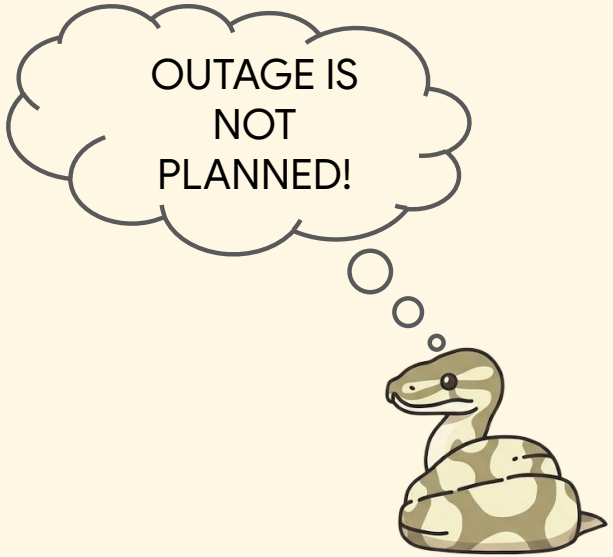
THE MAJORITY

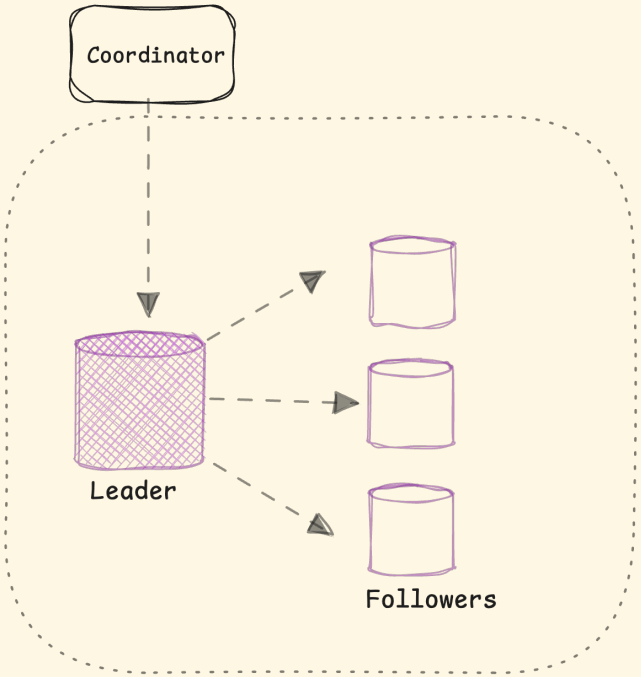


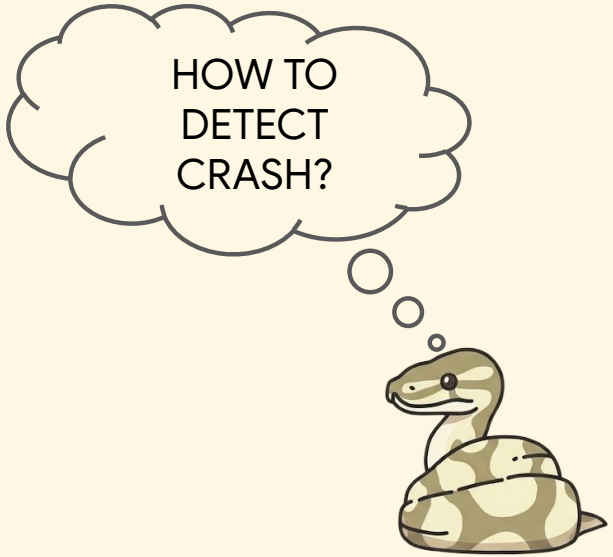

```
make checkpoint STAGE=07

make lab STAGE=07

kvwrite order paid

kvkill 1

kvwrite order shipped

kvread order
```

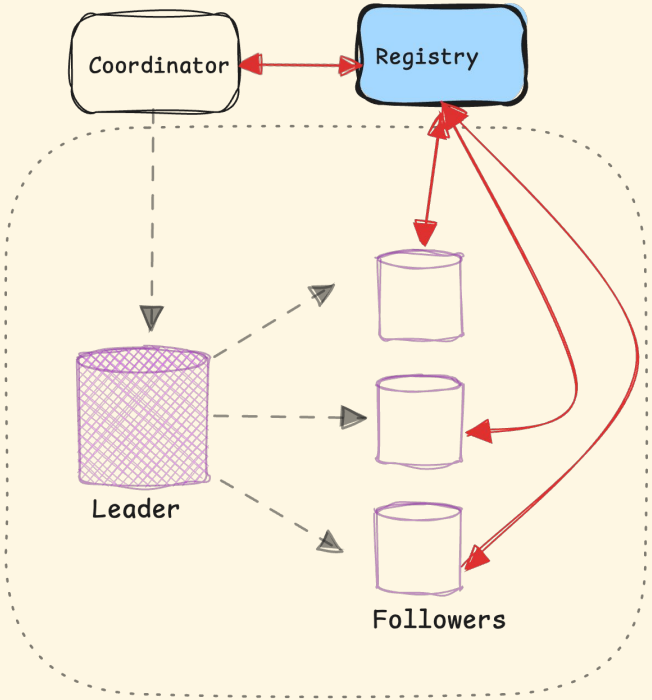
01 02 03 04 05 06 07 08 09 10


Coordinator
Leader
Followers


OUTAGE IS
NOT
PLANNED!

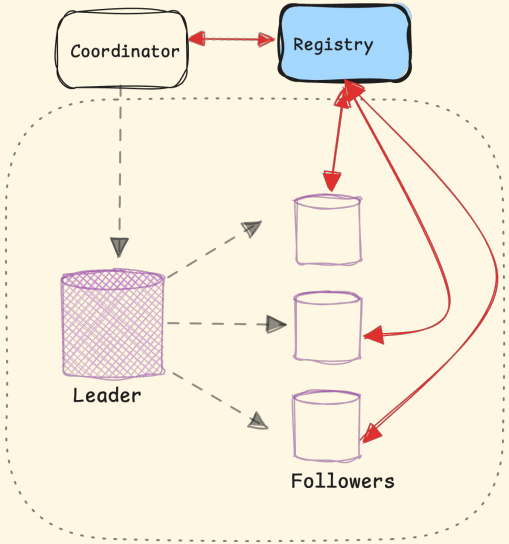

Coordinator
Leader
Followers


HOW TO DETECT CRASH?


Coordinator
Registry
Leader
Followers


HEARTBEATS

REGISTRY



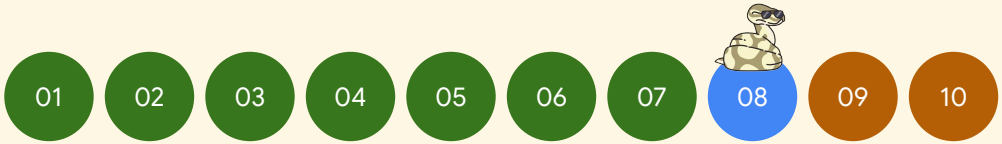

```
make todo STAGE=08

make lab STAGE=08

kvwrite order paid

kvcrash 1

kvwrite order shipped

kvstatus
```

01 02 03 04 05 06 07 08 09 10

REGISTRY ✏️

```python
def heartbeat_loop():
  """Send heartbeats to the registry so it knows this node is alive."""
```

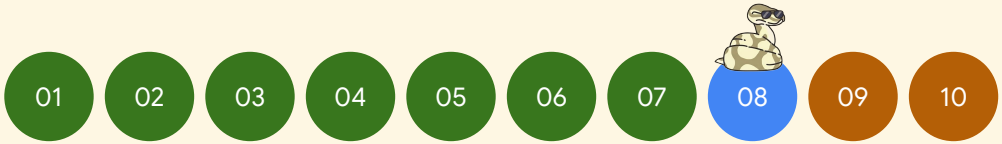
```
make todo STAGE=08

make lab STAGE=08

kvwrite order paid

kvcrash 1

kvwrite order shipped

kvstatus
```



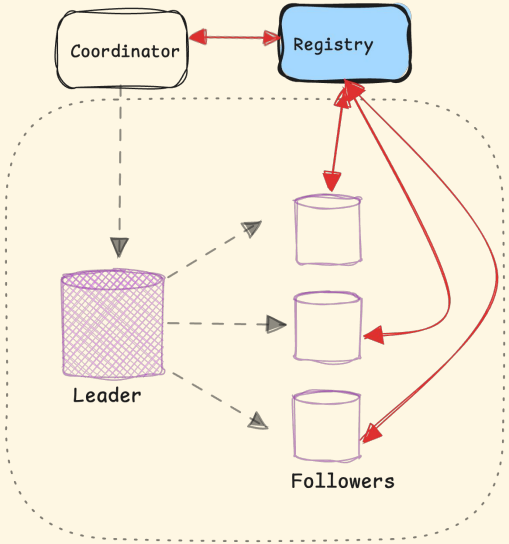
REGISTRY ✏️

```
resp = requests.post(
    f"{REGISTRY_URL}/heartbeat",
    json={
        "node_id": NODE_ID,
        "port": NODE_PORT,
        "url": f"http://localhost:{NODE_PORT}",
        "role": NODE_ROLE
    },
    timeout=2
)
```

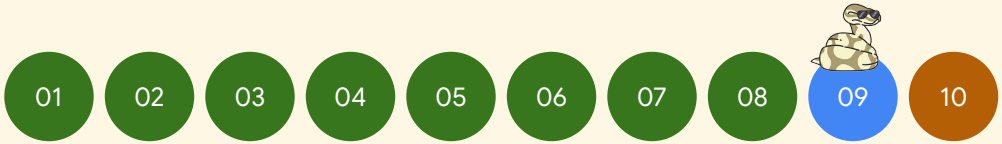
```
make checkpoint STAGE=08

make lab STAGE=08

kvwrite order paid

kvcrash 1

kvwrite order shipped

kvstatus
```



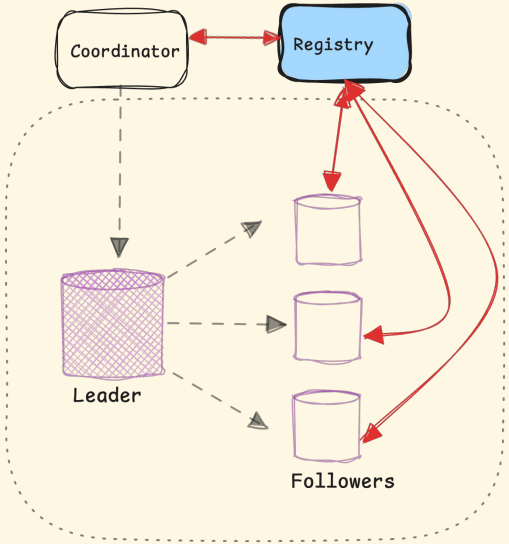
RECOVERY

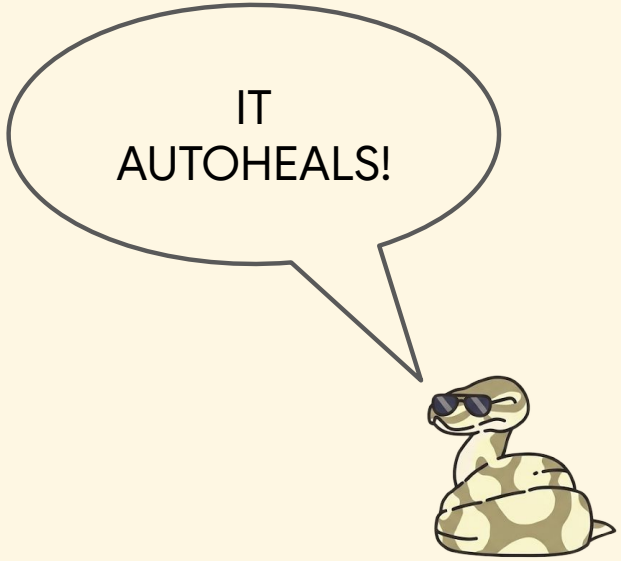


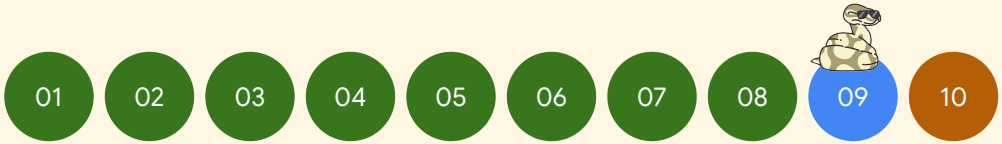

```
make checkpoint STAGE=09

make lab STAGE=09

kvwrite order paid

kvcrash 1

kvstatus
```



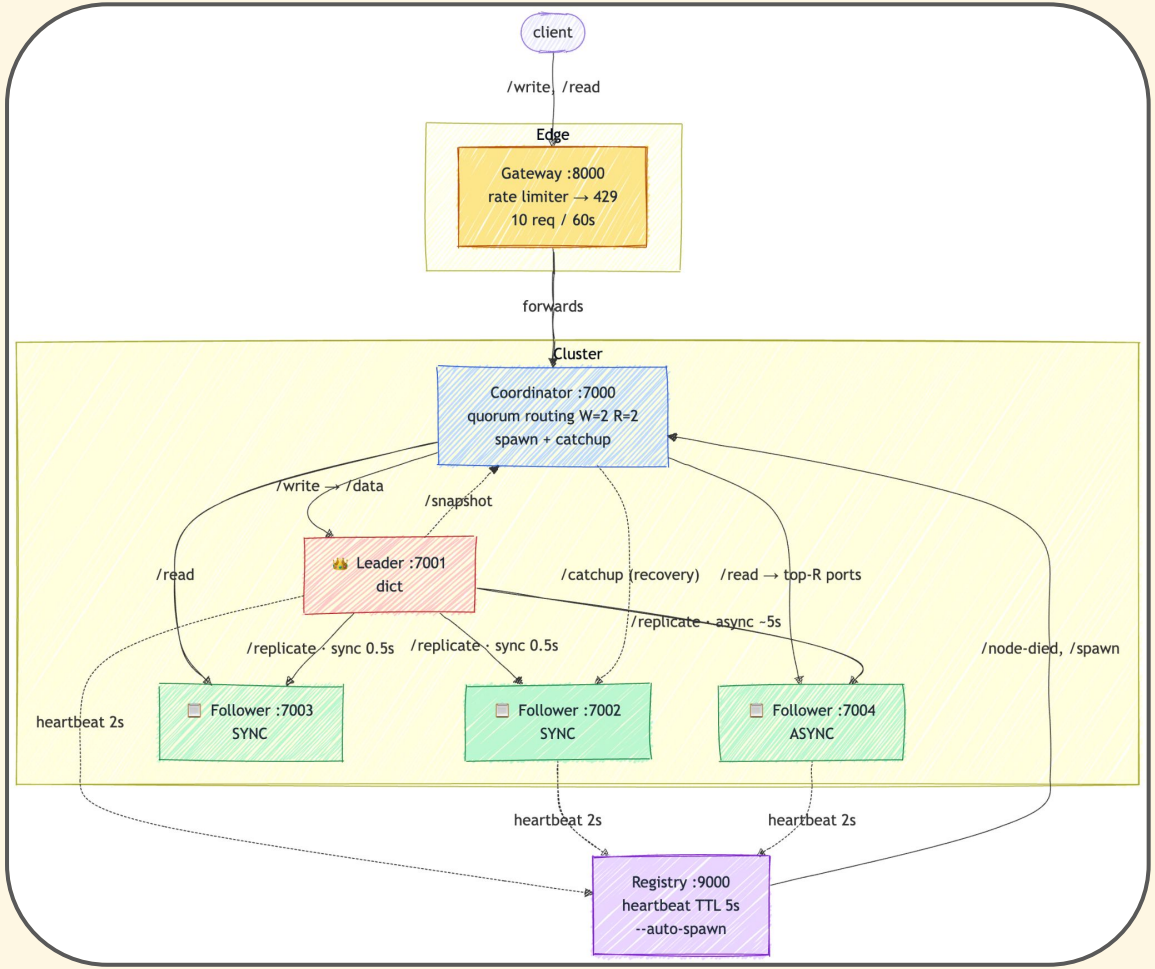
RECOVERY


Coordinator
Registry
Leader
Followers


IT
AUTOHEALS!


01
02
03
04
05
06
07
08
09
10

WE HAVE COME A LONG WAY!


client
/write, /read
Edge
Gateway :8000
rate limiter → 429
10 req / 60s
forwards
Cluster
Coordinator :7000
quorum routing W=2 R=2
spawn + catchup
/write → /data
/snapshot
/read
Leader :7001
dict
/catchup (recovery)
/read → top-R ports
/replicate - async ~5s
/replicate - sync 0.5s
/replicate - sync 0.5s
/node-died, /spawn
heartbeat 2s
Follower :7003
SYNC
Follower :7002
SYNC
Follower :7004
ASYNC
heartbeat 2s
heartbeat 2s
Registry :9000
heartbeat TTL 5s
--auto-spawn

THANK YOU!